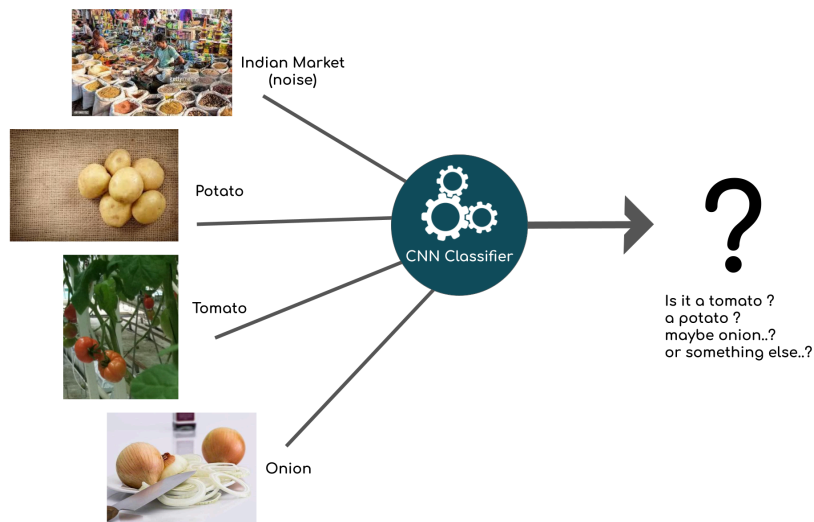


✓ Case Study : Ninjacart

[Ninjacart](#) is India's largest fresh produce supply chain company, connecting producers directly with the retailers.

- You are a Data Scientist at *Ninjacart*, you have curated a dataset consisting of some vegetables and other images at a vegetable market.
- You want to classify these images using computer vision.
- We will explore how to preprocess, load, augment our raw data and train state-of-the-art model architectures on it and compare them to analyze the best model for our use-case.

Problem Overview



1. Load data and perform EDA

✓ Importing required libraries

Importing the required libraries which we will be using throughout our training pipeline

```
import os
# os.environ["CUDA_VISIBLE_DEVICES"]="1"
import glob
import random
import pandas as pd
import seaborn as sns
import tensorflow as tf
import matplotlib.pyplot as plt
```

✓ Loading the Data - 10 Points

Downloading our Dataset from Google drive using `gdown` command

```
!gdown https://drive.google.com/uc?id=1clZX-lV_MLxKHSyeyTheX50C0tNCUcqT
```

```
📄 Downloading...
From (original): https://drive.google.com/uc?id=1clZX-lV\_MLxKHSyeyTheX50C0tNCUcqT
From (redirected): https://drive.google.com/uc?id=1clZX-lV\_MLxKHSyeyTheX50C0tNCUcqT&confirm=t&uuiid=58385fff-0d79-4baa-b4
To: /content/ninjacart_data.zip
100% 275M/275M [00:03<00:00, 80.9MB/s]
```

Unzipping our downloaded data

```
!unzip /content/ninjacart_data.zip
```



```

inflating: ninjacart_data/train/tomato/tomato854.png
inflating: ninjacart_data/train/tomato/tomato855.png
inflating: ninjacart_data/train/tomato/tomato856.png
inflating: ninjacart_data/train/tomato/tomato857.png
inflating: ninjacart_data/train/tomato/tomato858.png
inflating: ninjacart_data/train/tomato/tomato859.png
inflating: ninjacart_data/train/tomato/tomato86.png
inflating: ninjacart_data/train/tomato/tomato860.png
inflating: ninjacart_data/train/tomato/tomato861.png
inflating: ninjacart_data/train/tomato/tomato862.png
inflating: ninjacart_data/train/tomato/tomato863.png
inflating: ninjacart_data/train/tomato/tomato864.png
inflating: ninjacart_data/train/tomato/tomato865.png
inflating: ninjacart_data/train/tomato/tomato866.png
inflating: ninjacart_data/train/tomato/tomato867.png
inflating: ninjacart_data/train/tomato/tomato868.png
inflating: ninjacart_data/train/tomato/tomato869.png
inflating: ninjacart_data/train/tomato/tomato87.png
inflating: ninjacart_data/train/tomato/tomato870.png
inflating: ninjacart_data/train/tomato/tomato871.png
inflating: ninjacart_data/train/tomato/tomato872.png
inflating: ninjacart_data/train/tomato/tomato873.png
inflating: ninjacart_data/train/tomato/tomato874.png
inflating: ninjacart_data/train/tomato/tomato875.png
inflating: ninjacart_data/train/tomato/tomato876.png
inflating: ninjacart_data/train/tomato/tomato877.png
inflating: ninjacart_data/train/tomato/tomato878.png
inflating: ninjacart_data/train/tomato/tomato879.png
inflating: ninjacart_data/train/tomato/tomato88.png
inflating: ninjacart_data/train/tomato/tomato880.png
inflating: ninjacart_data/train/tomato/tomato881.png
inflating: ninjacart_data/train/tomato/tomato882.png
inflating: ninjacart_data/train/tomato/tomato883.png
inflating: ninjacart_data/train/tomato/tomato884.png
inflating: ninjacart_data/train/tomato/tomato885.png
inflating: ninjacart_data/train/tomato/tomato886.png
inflating: ninjacart_data/train/tomato/tomato887.png
inflating: ninjacart_data/train/tomato/tomato888.png
inflating: ninjacart_data/train/tomato/tomato889.png
inflating: ninjacart_data/train/tomato/tomato89.png
inflating: ninjacart_data/train/tomato/tomato890.png
inflating: ninjacart_data/train/tomato/tomato891.png
inflating: ninjacart_data/train/tomato/tomato892.png
inflating: ninjacart_data/train/tomato/tomato893.png
inflating: ninjacart_data/train/tomato/tomato894.png
inflating: ninjacart_data/train/tomato/tomato9.png
inflating: ninjacart_data/train/tomato/tomato90.png
inflating: ninjacart_data/train/tomato/tomato91.png
inflating: ninjacart_data/train/tomato/tomato92.png
inflating: ninjacart_data/train/tomato/tomato93.png
inflating: ninjacart_data/train/tomato/tomato94.png
inflating: ninjacart_data/train/tomato/tomato95.png
inflating: ninjacart_data/train/tomato/tomato96.png
inflating: ninjacart_data/train/tomato/tomato97.png
inflating: ninjacart_data/train/tomato/tomato98.png
inflating: ninjacart_data/train/tomato/tomato99.png

```

Let's analyze what data we have curated !

We can visualize from the data directory, the data is divided into train/test folders, consisting of four subfolders containing images of :

- Indian market
- Onion
- Potato
- Tomato

Directory structure of our Dataset

```

ninjacart_data
├── test
│   ├── indian market
│   ├── onion
│   ├── potato
│   └── tomato
└── train
    ├── indian market
    ├── onion
    ├── potato
    └── tomato

```

✓ Visualizing our Images

Now let's plot some random images from our Dataset from each class !

```
train_folder = '/content/ninjacart_data/train'

images = []
for folder in os.listdir(train_folder):
    for image in os.listdir(train_folder + '/' + folder):
        images.append(os.path.join(train_folder, folder, image))

fig = plt.figure(1, figsize=(15, 9))
fig.suptitle('Data overview', fontsize=25)
plt.axis('off')
n = 0
for i in range(16):
    n += 1
    random_img = random.choice(images)
    imgs = tf.keras.utils.load_img(random_img)
    plt.subplot(4, 4, n)
    plt.axis('off')
    plt.imshow(imgs)

plt.show()
```



Data overview



✓ Exploratory Data Analysis - 20 points

- We write a code snippet to fetch the paths of all images for every class in our train and test set in root folder, which we will use later to test our model on random samples.
- The image arrays and count for every class are stored in a dictionary to plot its distribution

```
class_dirs = os.listdir("/content/ninjacart_data/train") # list all directories inside "train" folder
image_dict = {} # dict to store image array(key) for every class(value)
```

```

count_dict = {} # dict to store count of files(key) for every class(value)

# iterate over all class_dirs
for cls in class_dirs:
    # get list of all paths inside the subdirectory
    file_paths = glob.glob(f'/content/ninjacart_data/train/{cls}/*')
    # count number of files in each class and add it to count_dict
    count_dict[cls] = len(file_paths)
    # select random item from list of image paths
    image_path = random.choice(file_paths)
    # load image using keras utility function and save it in image_dict
    image_dict[cls] = tf.keras.utils.load_img(image_path)

```

Store image arrays and count for each class in test data :

```

class_dirs_test = os.listdir("/content/ninjacart_data/test") # list all directories inside "test" folder
image_dict_test = {} # dict to store image array(key) for every class(value)
count_dict_test = {} # dict to store count of files(key) for every class(value)

# iterate over all class_dirs
for cls in class_dirs_test:
    # get list of all paths inside the subdirectory
    file_paths_test = glob.glob(f'/content/ninjacart_data/test/{cls}/*')
    # count number of files in each class and add it to count_dict
    count_dict_test[cls] = len(file_paths_test)
    # select random item from list of image paths
    image_path = random.choice(file_paths_test)
    # load image using keras utility function and save it in image_dict
    image_dict_test[cls] = tf.keras.utils.load_img(image_path)

```

✓ Image Dimensions Analysis

Plot a Grid of training samples from each class to display their dimensions

```

# Visualize Random Sample from each class

plt.figure(figsize=(15, 7))
# iterate over dictionary items (class label, image array)
for i, (cls,img) in enumerate(image_dict.items()):
    # create a subplot axis
    ax = plt.subplot(1, 4, i + 1)
    # plot each image
    plt.imshow(img)
    # set "class name" along with "image size" as title
    plt.title(f'{cls}, {img.size}')
    plt.axis("off")

```



- Analyze how our Dataset consists of uneven sized images !
- We have some images of the market, which we will label as class 'noise'
- To feed this data into our models, we need to preprocess it

✓ Class Distribution Analysis:-

- Let's visualize how many samples of each class we have and if there's any class imbalance or not

- Plotting our Class distributions using our image_dict and count_dict which stores the images for each class along with its count
- We also feed our count_dict to a Dataframe to print the count for each class in our training and test directories

Let's now Plot the Data Distribution of Training Data across Classes

```
df_count_train = pd.DataFrame({
    "class": count_dict.keys(),      # keys of count_dict are class labels
    "count": count_dict.values(),    # value of count_dict contain counts of each class
})
```

```
print("Count of training samples per class:\n", df_count_train, "\n")
```

draw a bar plot using pandas in-built plotting function

```
b = df_count_train.plot.bar(x='class', y='count', title="Training Data Count per class", label='train sample count', color='')
```

Annotate the bars in our plot with their values

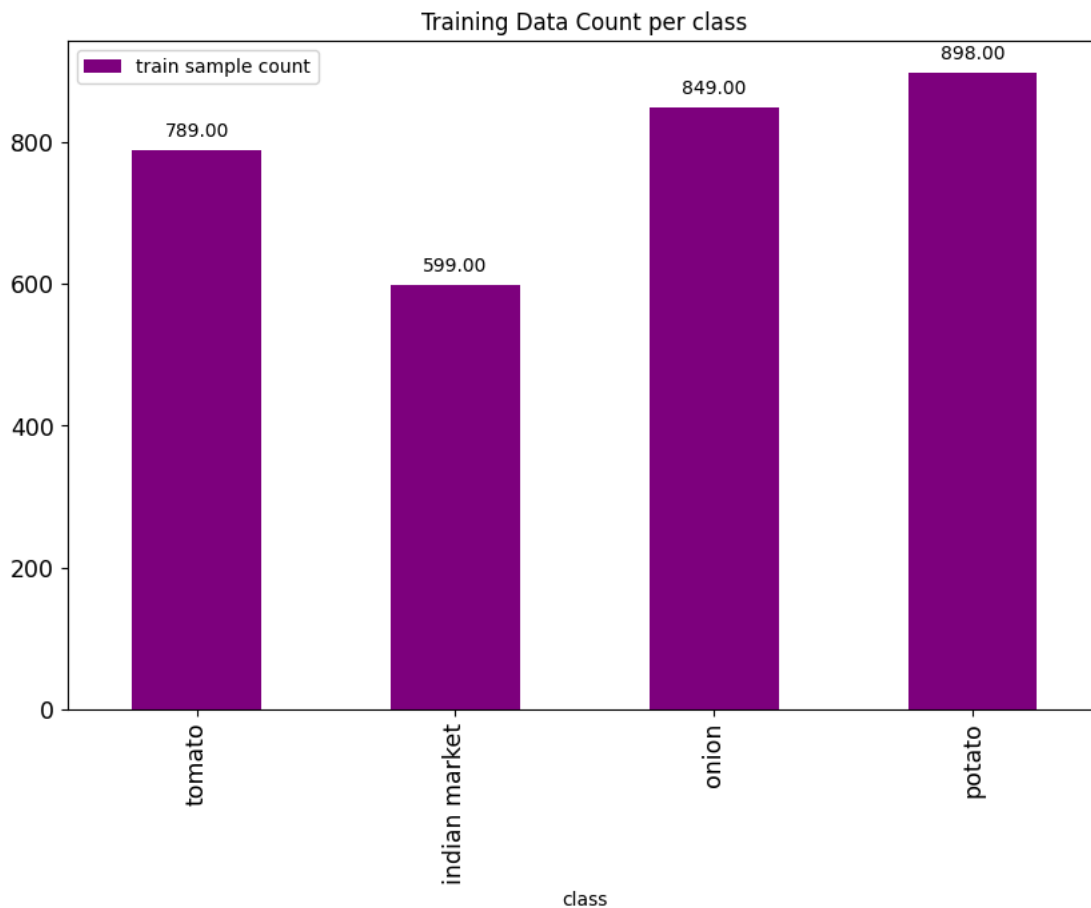
for q in b.patches:

```
    b.annotate(f'{q.get_height():0.2f}', (q.get_x() + q.get_width() / 2., q.get_height()),
               ha = 'center', va = 'center', xytext = (0, 10), textcoords = 'offset points')
```

```
plt.show()
```

↻ Count of training samples per class:

	class	count
0	tomato	789
1	indian market	599
2	onion	849
3	potato	898



- From the above plot, we can quickly analyze that our Data is not evenly distributed, but is also not imbalanced, so we are good to go !
- But what about the minority class (*indian market*) ?
 - We will use some {techniques} before feeding images to our network so that our model is generalized and is not biased for any class !

Let's now Plot the Data Distribution of Training Data across Classes

```
df_count_test = pd.DataFrame({
    "class": count_dict_test.keys(),      # keys of count_dict are class labels
    "count": count_dict_test.values(),    # value of count_dict contain counts of each class
})
```

```
print("Count of testing samples per class:\n", df_count_test, "\n")
```

draw a bar plot using pandas in-built plotting function

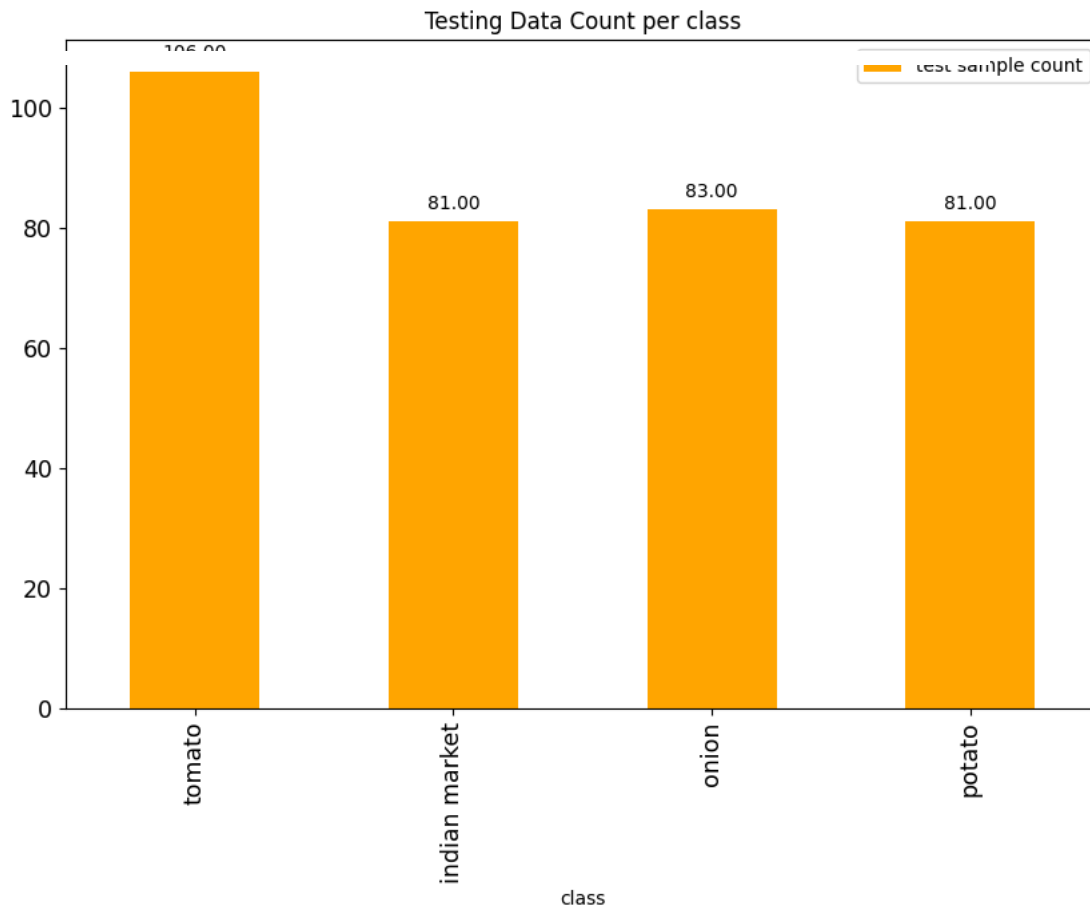
```

a = df_count_test.plot.bar(x='class', y='count', title="Testing Data Count per class", color='orange', label='test sample cc
for p in a.patches:
    a.annotate(f'{p.get_height():0.2f}', (p.get_x() + p.get_width() / 2., p.get_height()),
               ha = 'center', va = 'center', xytext = (0, 10), textcoords = 'offset points')
plt.show()

```

Count of testing samples per class:

	class	count
0	tomato	106
1	indian market	81
2	onion	83
3	potato	81



✓ Preparing the Dataset

Now that we have analyzed and Explored our Raw Data, now to feed our Data into our model,

- Every image needs to be labelled according to its class and converted to a tensor.
- We will split our data into 3 sets : Train, Test and Validation

For this task, we will use `tf.keras.utils.image_dataset_from_directory`, which will :

- Create an **(image, label)** pair for every image
- Automatically label the images using one-hot encoding
 - Eg : An image in the folder *indian market* will be labelled as **[1, 0, 0, 0]**
 - Since we have 4 classes, our classes will be one-hot encoded **alphabetically**,
 - meaning our first class is indian market, fourth will be tomato.
- Batching (Grouping) images together according to *batch_size*
- Resize the Images to the shape **(256, 256)**
- Seed the shuffling of our images, so that it shuffles in a 'fixed' way
 - which will be used to create our validation data
- Splitting 20% of our training data for validation
- Shuffling the data according to the seed

```

train_dir = '/content/ninjacart_data/train'
test_dir = '/content/ninjacart_data/test'

```

```
# Specifying the image size to resize all images
image_size = (256, 256)
```

```
train_ds = tf.keras.utils.image_dataset_from_directory(directory = train_dir,
                                                    label_mode = 'categorical',
                                                    batch_size = 32,
                                                    image_size = image_size,
                                                    seed = 2022,
                                                    validation_split = 0.2,
                                                    subset = "training",
                                                    shuffle=True)
```

➦ Found 3135 files belonging to 4 classes.
Using 2508 files for training.

```
test_ds = tf.keras.utils.image_dataset_from_directory(directory = test_dir,
                                                    label_mode = 'categorical',
                                                    batch_size = 32,
                                                    image_size = image_size,
                                                    seed = 2022,
                                                    )
```

➦ Found 351 files belonging to 4 classes.

Specifying our Validation data, which we took from our train_ds specifying `validation_split = 0.2`

We take the last 20% data from our train_ds, we use the same seed as train_ds

```
valid_ds = tf.keras.utils.image_dataset_from_directory(directory = train_dir,
                                                    label_mode = 'categorical',
                                                    batch_size = 32,
                                                    image_size = image_size,
                                                    seed = 2022,
                                                    validation_split = 0.2,
                                                    subset = "validation")
```

➦ Found 3135 files belonging to 4 classes.
Using 627 files for validation.

Since `label_mode = 'categorical'` one-hot encodes the classes **alphabetically**,
indian market (which we consider noise) = class 1 and *tomato* = class 4

```
# index      0      1      2      3
class_names = ['noise', 'onion', 'potato', 'tomato']
```

✓ Creating Model architecture and training :

✓ Building custom CNN model- 10 Points

Let's build a baseline CNN Architecture with some Convolutional blocks which MaxPooling

✓ Defining the model architecture

```

model = tf.keras.Sequential([

tf.keras.layers.Rescaling(1./255),
# tf.keras.layers.InputLayer(input_shape = [image_size[0], image_size[1], 3]),

tf.keras.layers.Conv2D(filters = 32, kernel_size = (3,3), padding = 'Same', activation = 'relu'),
tf.keras.layers.Conv2D(filters = 32, kernel_size = (3,3), padding = 'Same', activation = 'relu'),
tf.keras.layers.MaxPool2D(pool_size = (2,2)),

tf.keras.layers.Conv2D(filters = 64, kernel_size = (3,3), padding = 'Same', activation = 'relu'),
tf.keras.layers.Conv2D(filters = 64, kernel_size = (3,3), padding = 'Same', activation = 'relu'),
tf.keras.layers.MaxPool2D(pool_size = (2,2)),

tf.keras.layers.Conv2D(filters = 128, kernel_size = (3,3), padding = 'Same', activation = 'relu'),

tf.keras.layers.GlobalAveragePooling2D(),
tf.keras.layers.Dense(4, activation = 'softmax')
])

```

✓ Compiling the model

```

model.compile(optimizer = tf.keras.optimizers.Adam(learning_rate = 1e-4),
              loss = tf.keras.losses.CategoricalCrossentropy(),
              metrics = ['accuracy', 'Precision', 'Recall'])

```

✓ Model Training

- Implementing TensorBoard callback to plot our model's metrics later and store them in our log directory

```

log_dir_1 = "logs/Custom_CNN_overfit"
tensorboard_cb = tf.keras.callbacks.TensorBoard(log_dir=log_dir_1, histogram_freq=1)

```

```

%load_ext tensorboard
log_folder = log_dir_1

```

```

history = model.fit(train_ds, epochs=20, validation_data=valid_ds, callbacks=[tensorboard_cb])

```

```

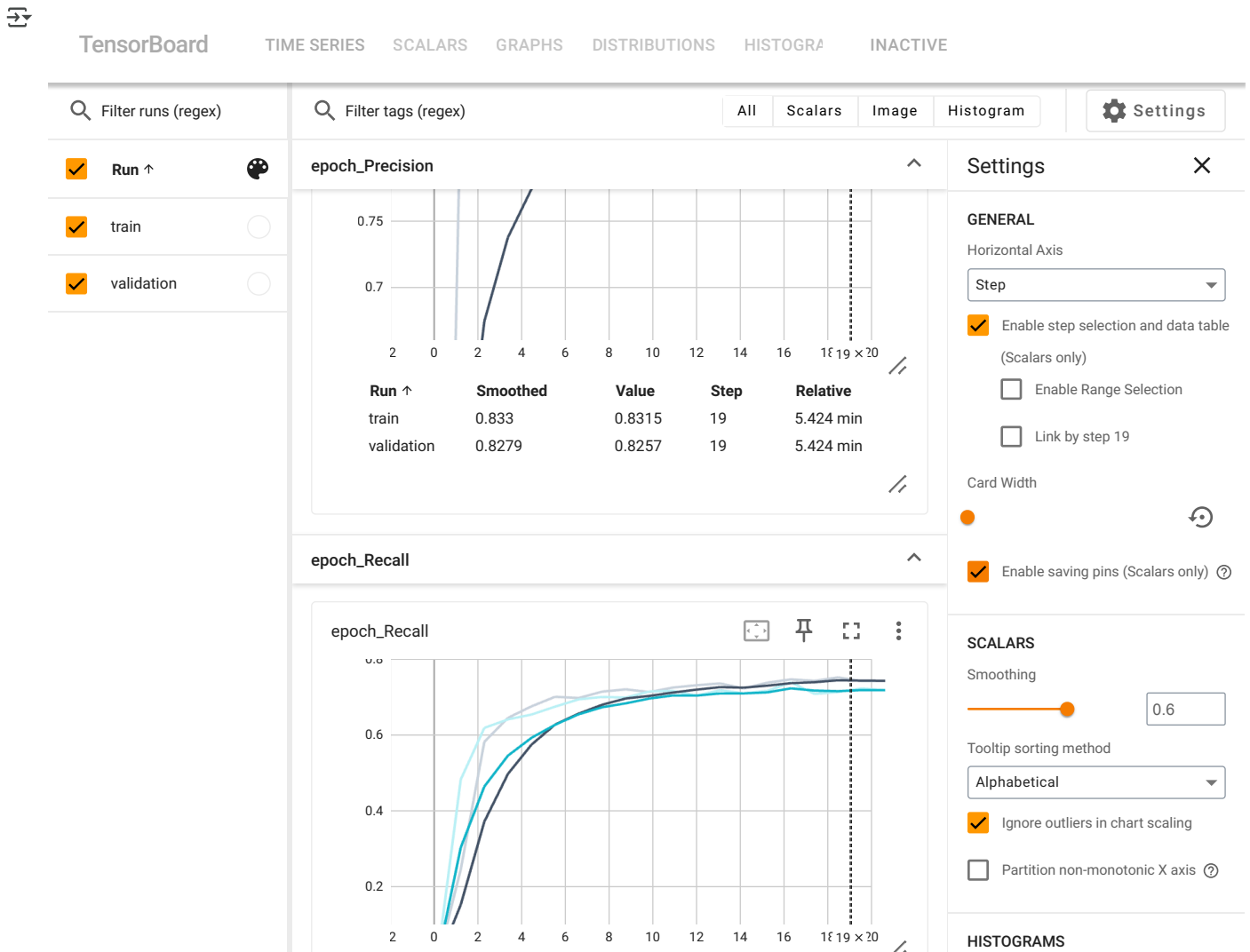
➡ Epoch 1/20
79/79 ━━━━━━━━━━━ 44s 333ms/step - Precision: 0.0000e+00 - Recall: 0.0000e+00 - accuracy: 0.2948 - loss: 1.3571
Epoch 2/20
79/79 ━━━━━━━━━━━ 12s 152ms/step - Precision: 0.8738 - Recall: 0.1137 - accuracy: 0.5511 - loss: 1.1246 - val_P
Epoch 3/20
79/79 ━━━━━━━━━━━ 12s 156ms/step - Precision: 0.8229 - Recall: 0.5384 - accuracy: 0.7146 - loss: 0.7721 - val_P
Epoch 4/20
79/79 ━━━━━━━━━━━ 21s 160ms/step - Precision: 0.8196 - Recall: 0.6450 - accuracy: 0.7535 - loss: 0.6788 - val_P
Epoch 5/20
79/79 ━━━━━━━━━━━ 21s 164ms/step - Precision: 0.8239 - Recall: 0.6595 - accuracy: 0.7459 - loss: 0.6456 - val_P
Epoch 6/20
79/79 ━━━━━━━━━━━ 12s 148ms/step - Precision: 0.8395 - Recall: 0.7083 - accuracy: 0.7731 - loss: 0.6221 - val_P
Epoch 7/20
79/79 ━━━━━━━━━━━ 22s 163ms/step - Precision: 0.8345 - Recall: 0.6978 - accuracy: 0.7806 - loss: 0.6091 - val_P
Epoch 8/20
79/79 ━━━━━━━━━━━ 21s 163ms/step - Precision: 0.8359 - Recall: 0.7229 - accuracy: 0.7891 - loss: 0.5811 - val_P
Epoch 9/20
79/79 ━━━━━━━━━━━ 21s 166ms/step - Precision: 0.8395 - Recall: 0.7314 - accuracy: 0.7859 - loss: 0.5938 - val_P
Epoch 10/20
79/79 ━━━━━━━━━━━ 19s 150ms/step - Precision: 0.8345 - Recall: 0.7221 - accuracy: 0.7828 - loss: 0.5586 - val_P
Epoch 11/20
79/79 ━━━━━━━━━━━ 21s 152ms/step - Precision: 0.8375 - Recall: 0.7251 - accuracy: 0.7917 - loss: 0.5487 - val_P
Epoch 12/20
79/79 ━━━━━━━━━━━ 13s 162ms/step - Precision: 0.8354 - Recall: 0.7377 - accuracy: 0.7865 - loss: 0.5442 - val_P
Epoch 13/20
79/79 ━━━━━━━━━━━ 12s 151ms/step - Precision: 0.8422 - Recall: 0.7394 - accuracy: 0.7931 - loss: 0.5360 - val_P
Epoch 14/20
79/79 ━━━━━━━━━━━ 22s 165ms/step - Precision: 0.8469 - Recall: 0.7358 - accuracy: 0.7970 - loss: 0.5443 - val_P
Epoch 15/20
79/79 ━━━━━━━━━━━ 20s 164ms/step - Precision: 0.8475 - Recall: 0.7422 - accuracy: 0.8007 - loss: 0.5281 - val_P
Epoch 16/20
79/79 ━━━━━━━━━━━ 12s 151ms/step - Precision: 0.8430 - Recall: 0.7492 - accuracy: 0.8059 - loss: 0.5067 - val_P
Epoch 17/20
79/79 ━━━━━━━━━━━ 12s 152ms/step - Precision: 0.8457 - Recall: 0.7505 - accuracy: 0.7973 - loss: 0.5077 - val_P
Epoch 18/20
79/79 ━━━━━━━━━━━ 13s 168ms/step - Precision: 0.8426 - Recall: 0.7511 - accuracy: 0.8007 - loss: 0.5128 - val_P
Epoch 19/20
79/79 ━━━━━━━━━━━ 21s 168ms/step - Precision: 0.8332 - Recall: 0.7474 - accuracy: 0.7932 - loss: 0.5469 - val_P
Epoch 20/20
79/79 ━━━━━━━━━━━ 20s 162ms/step - Precision: 0.8375 - Recall: 0.7461 - accuracy: 0.7976 - loss: 0.5016 - val_P

```

```

%tensorboard --logdir={log_folder} --port=6007

```



✓ Plotting our Model's performance

```
def plot_acc_loss(model_history, n_epochs):

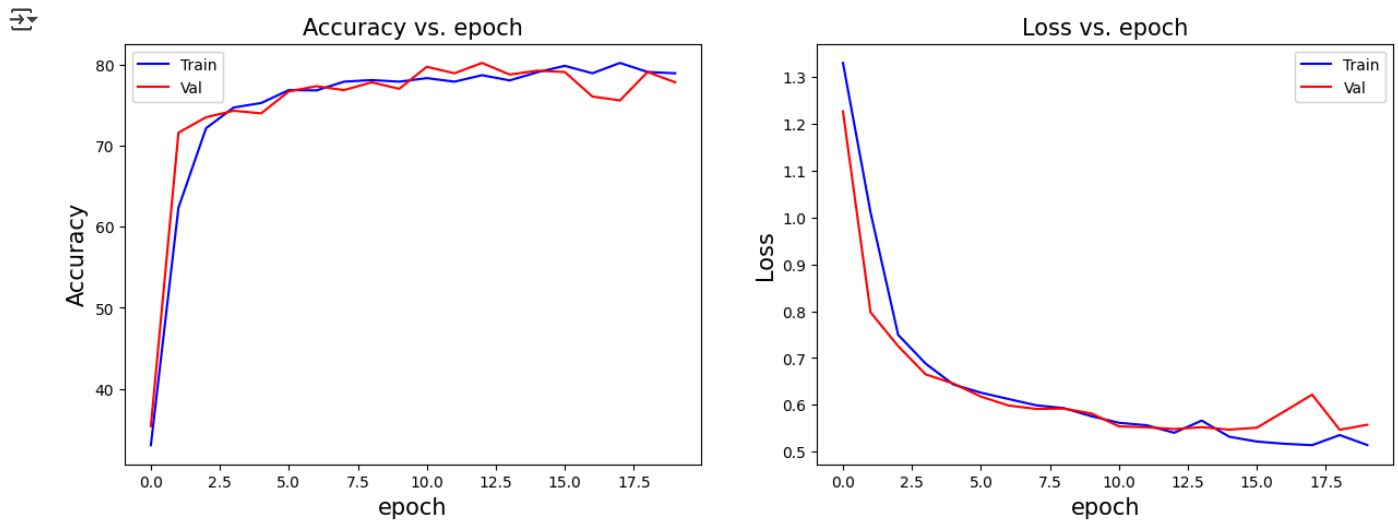
    fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(15,5))
    ax = axes.ravel()

    #accuracy graph
    ax[0].plot(range(0, n_epochs), [acc * 100 for acc in model_history.history['accuracy']], label='Train', color='b')
    ax[0].plot(range(0, n_epochs), [acc * 100 for acc in model_history.history['val_accuracy']], label='Val', color='r')
    ax[0].set_title('Accuracy vs. epoch', fontsize=15)
    ax[0].set_ylabel('Accuracy', fontsize=15)
    ax[0].set_xlabel('epoch', fontsize=15)
    ax[0].legend()

    #loss graph
    ax[1].plot(range(0, n_epochs), model_history.history['loss'], label='Train', color='b')
    ax[1].plot(range(0, n_epochs), model_history.history['val_loss'], label='Val', color='r')
    ax[1].set_title('Loss vs. epoch', fontsize=15)
    ax[1].set_ylabel('Loss', fontsize=15)
    ax[1].set_xlabel('epoch', fontsize=15)
    ax[1].legend()

    #display the graph
    plt.show()

plot_acc_loss(history, n_epochs = 20)
```



```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 256, 256, 3)	0
conv2d (Conv2D)	(None, 256, 256, 32)	896
conv2d_1 (Conv2D)	(None, 256, 256, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 128, 128, 32)	0
conv2d_2 (Conv2D)	(None, 128, 128, 64)	18,496
conv2d_3 (Conv2D)	(None, 128, 128, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 64, 64, 64)	0
conv2d_4 (Conv2D)	(None, 64, 64, 128)	73,856
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0
dense (Dense)	(None, 4)	516

Total params: 419,822 (1.60 MB)
Trainable params: 139,940 (546.64 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 279,882 (1.07 MB)

✓ Testing the model

We list all the files in our test directory and append their complete paths to a list `test_images` which we will use to test our model by predicting on on random unseen data

```
test_images = []

for folder in os.listdir(test_dir):
    for image in os.listdir(test_dir + '/' + folder):
        test_images.append(os.path.join(test_dir, folder, image))
```

✓ Random Sample testing

To test our model on a random sample :

- Read a random image from `test_images` list containing paths of all test images
- Resize an image to (256, 256)
- Add an extra dimension to our image's first dimension index
- Predict on the image using `model.predict()`


```
def grid_test_model(model_name):

    fig = plt.figure(1, figsize=(17, 11))
    plt.axis('off')
    n = 0
    for i in range(8):
        n += 1

        img_0 = tf.keras.utils.load_img(random.choice(test_images))
        img_0 = tf.keras.utils.img_to_array(img_0)
        img_0 = tf.image.resize(img_0, (256, 256))
        img_1 = tf.expand_dims(img_0, axis = 0)

        pred = model_name.predict(img_1)
        predicted_label = tf.argmax(pred, 1).numpy().item()

        for item in pred :
            item = tf.round((item*100))

        plt.subplot(2, 4, n)
        plt.axis('off')
        plt.title(f'prediction : {class_names[predicted_label]}\n\n'
                  f'{item[0]} % {class_names[0]}\n'
                  f'{item[1]} % {class_names[1]}\n'
                  f'{item[2]} % {class_names[2]}\n'
                  f'{item[3]} % {class_names[3]}\n')
        plt.imshow(img_0/255)
    plt.show()

grid_test_model(model)
```

```

1/1 _____ 1s 987ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 29ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 29ms/step
1/1 _____ 0s 29ms/step

```

prediction : potato

2.0 % noise
6.0 % onion
92.0 % potato
0.0 % tomato

prediction : noise

46.0 % noise
22.0 % onion
32.0 % potato
0.0 % tomato

prediction : noise

95.0 % noise
2.0 % onion
3.0 % potato
0.0 % tomato

prediction : noise

58.0 % noise
42.0 % onion
0.0 % potato
0.0 % tomato



prediction : onion

4.0 % noise
91.0 % onion
5.0 % potato
0.0 % tomato



prediction : tomato

0.0 % noise
0.0 % onion
0.0 % potato
100.0 % tomato



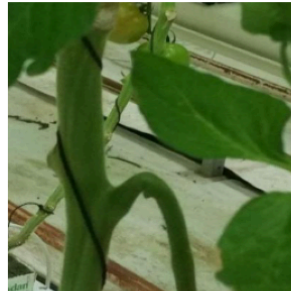
prediction : potato

19.0 % noise
16.0 % onion
65.0 % potato
0.0 % tomato



prediction : onion

6.0 % noise
61.0 % onion
33.0 % potato
0.0 % tomato



By observing the above plot, we can analyze that our Model is **not too sure** of the correct class....

We have to revamp our model to increase its confidence

Also, notice how our images are evenly sized (256x256) now !

✓ Evaluating accuracy on each class

First we specify paths of each of our test classes

- We define a function `classwise_accuracy` which calculates accuracy for each class by taking in class path and model,
- We append all the file paths for the input class in a list
- from which we predict on all images and check if the label and predicted label are equal, according to which we calculate the accuracy of our model on the input class

```

noise_path = '/content/ninjacart_data/test/indian market'
onion_path = '/content/ninjacart_data/test/onion'
potato_path = '/content/ninjacart_data/test/potato'
tomato_path = '/content/ninjacart_data/test/tomato'

```

```
def classwise_accuracy(class_path, class_name, model_name) :
    paths = []
    for i in os.listdir(class_path):
        paths.append(class_path + "/" + str(i))

    correct = 0
    total = 0

    for i in range(len(paths)):
        total+= 1

        img = tf.keras.utils.load_img(paths[i])
        img = tf.keras.utils.img_to_array(img)
        img = tf.image.resize(img, (256, 256))
        img = tf.expand_dims(img, axis = 0)

        pred = model_name.predict(img)
        if tf.argmax(pred[0]) == class_names.index(f"{class_name}"):
            correct+= 1

    print(f"Accuracy for class {class_name} is {round((correct/total)*100, 2)}% consisting of {len(paths)} images")

classwise_accuracy(noise_path, 'noise', model)
classwise_accuracy(onion_path, 'onion', model)
classwise_accuracy(potato_path, 'potato', model)
classwise_accuracy(tomato_path, 'tomato', model)
```

```
1/1 _____ 0s 33ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 35ms/step
Accuracy for class tomato is 98.11% consisting of 106 images
```

✓ Plotting the Confusion Matrix :

- We define four empty lists, in which we will store predicted classes by our model for a particular class images
- We define a function `conf_mat` which takes in the path of the class, empty list and the model
- The function reads an image from the given path, resizes it and converts to RGB
- Add an extra dimension at first index of the image and predict on the image with the input model
- Check which class the model has predicted the image belongs to and
- append +1 to the index of the class
 - Eg : If the model predicts that the image belongs to class tomato
 - we will append +1 at the fourth index(3) of our list like [0, 0, 0, 1]
- After calculating the value in our 4 lists for each class, we combine them, which gives us the confusion matrix of our model and plot them with seaborn

```
noise, tomato, potato, onion = 0, 0, 0, 0
```

```
l1 = []
l2 = []
l3 = []
l4 = []
```

```
sns.set(rc={'figure.figsize':(11.7, 8.27)})
sns.set(font_scale=1.2)
```

```
def conf_mat(class_path, pred_list, model_name) :
    noise, tomato, potato, onion = 0, 0, 0, 0
    for i in os.listdir(class_path):

        img = tf.keras.utils.load_img(class_path + "/" + str(i))
        img = tf.keras.utils.img_to_array(img)
        img = tf.image.resize(img, (256, 256))
        img = tf.expand_dims(img, axis = 0)

        pred = model_name.predict(img)
        predicted = tf.argmax(pred, 1).numpy().item()

        if predicted == 0:
            noise+= 1
        elif predicted == 1:
            onion+= 1
        elif predicted == 2:
            potato+= 1
        else:
            tomato+= 1

    for item in noise, onion, potato, tomato :
        pred_list.append(item)

conf_mat(noise_path, l1, model)
conf_mat(onion_path, l2, model)
conf_mat(potato_path, l3, model)
conf_mat(tomato_path, l4, model)

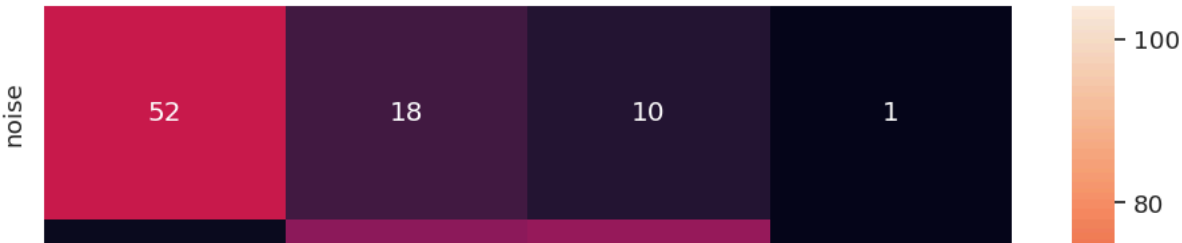
ax = sns.heatmap([l1, l2, l3, l4], xticklabels=class_names, yticklabels=class_names, annot=True, fmt='g')
ax.set(xlabel='Predicted label', ylabel='True label')
plt.show()
```

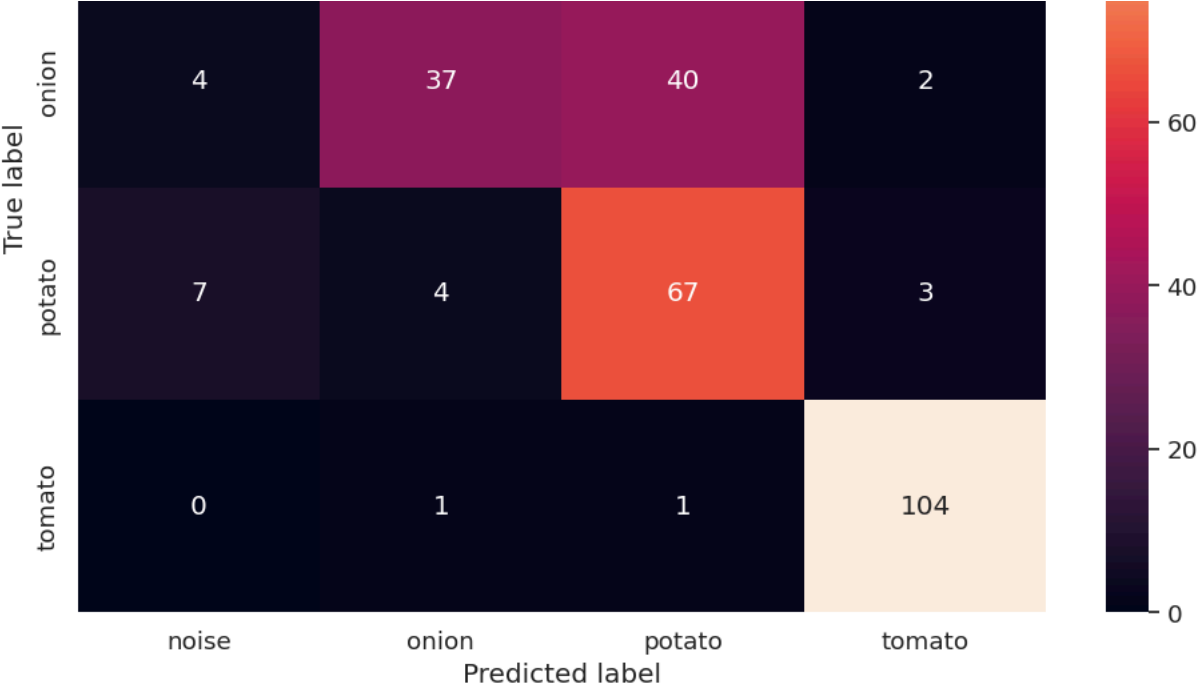
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 29ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 33ms/step
1/1 0s 32ms/step
1/1 0s 29ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 30ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 31ms/step
1/1 0s 33ms/step
1/1 0s 30ms/step
1/1 0s 33ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 34ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 45ms/step
1/1 0s 46ms/step
1/1 0s 46ms/step
1/1 0s 44ms/step
1/1 0s 43ms/step
1/1 0s 42ms/step
1/1 0s 44ms/step
1/1 0s 41ms/step
1/1 0s 44ms/step
1/1 0s 44ms/step
1/1 0s 44ms/step
1/1 0s 44ms/step
1/1 0s 43ms/step
1/1 0s 45ms/step
1/1 0s 43ms/step
1/1 0s 44ms/step
1/1 0s 46ms/step
1/1 0s 43ms/step
1/1 0s 43ms/step
1/1 0s 42ms/step
1/1 0s 47ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 38ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 32ms/step
1/1 0s 35ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 34ms/step
1/1 0s 39ms/step
1/1 0s 33ms/step
1/1 0s 30ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 29ms/step
1/1 0s 36ms/step
1/1 0s 31ms/step

1/1 _____ 0s 30ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 30ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 55ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 46ms/step

```
-- 1/1 0s 45ms/step
1/1 0s 41ms/step
1/1 0s 42ms/step
1/1 0s 40ms/step
1/1 0s 50ms/step
1/1 0s 43ms/step
1/1 0s 58ms/step
1/1 0s 50ms/step
1/1 0s 51ms/step
1/1 0s 54ms/step
1/1 0s 52ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 35ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 35ms/step
1/1 0s 31ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 34ms/step
1/1 0s 31ms/step
1/1 0s 36ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 39ms/step
1/1 0s 53ms/step
1/1 0s 46ms/step
1/1 0s 109ms/step
1/1 0s 55ms/step
1/1 0s 67ms/step
1/1 0s 58ms/step
1/1 0s 122ms/step
1/1 0s 43ms/step
1/1 0s 69ms/step
1/1 0s 96ms/step
1/1 0s 57ms/step
1/1 0s 53ms/step
1/1 0s 73ms/step
1/1 0s 97ms/step
1/1 0s 58ms/step
1/1 0s 61ms/step
1/1 0s 63ms/step
1/1 0s 51ms/step
1/1 0s 106ms/step
1/1 0s 46ms/step
1/1 0s 82ms/step
1/1 0s 103ms/step
1/1 0s 51ms/step
1/1 0s 91ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 32ms/step
1/1 0s 30ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 34ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 33ms/step
1/1 0s 35ms/step
```

1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 34ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 48ms/step
1/1 0s 49ms/step
1/1 0s 50ms/step
1/1 0s 51ms/step
1/1 0s 48ms/step
1/1 0s 51ms/step
1/1 0s 47ms/step
1/1 0s 46ms/step
1/1 0s 46ms/step
1/1 0s 52ms/step
1/1 0s 46ms/step
1/1 0s 43ms/step
1/1 0s 40ms/step
1/1 0s 61ms/step
1/1 0s 45ms/step
1/1 0s 51ms/step
1/1 0s 46ms/step
1/1 0s 35ms/step
1/1 0s 32ms/step
1/1 0s 35ms/step
1/1 0s 31ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 35ms/step
1/1 0s 31ms/step
1/1 0s 33ms/step
1/1 0s 32ms/step
1/1 0s 40ms/step
1/1 0s 34ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 35ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 36ms/step
1/1 0s 31ms/step
1/1 0s 36ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 37ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 34ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 33ms/step
1/1 0s 34ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 32ms/step
1/1 0s 35ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step





```
result = model.evaluate(test_ds)
dict(zip(model.metrics_names, result))
```

```
11/11 ————— 4s 424ms/step - Precision: 0.7682 - Recall: 0.7064 - accuracy: 0.7273 - loss: 0.6042
{'loss': 0.5619955658912659, 'compile_metrics': 0.7435897588729858}
```

- We can observe that our baseline CNN model has a categorical cross entropy loss of ~ 0.5
- which indicates overfitting, now let's work on how we can improve our model's performance by using some certain techniques !

✓ Model Tuning

✓ Augmenting our Data

- Data Augmentation introduces our model to different aspects and visualizations of class images, therefore increasing model generalization
- We apply 3 types of augmentation on our training dataset
 - RandomFlip : randomly flips images horizontally and vertically during training
 - RandomRotation : randomly rotates images during training by a factor
 - RandomTranslation : randomly translates images during training by a shift factor

```
augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal_and_vertical"),
    tf.keras.layers.RandomRotation(0.2),
    tf.keras.layers.RandomTranslation(height_factor = 0.2, width_factor=0.2)
])
```

```
aug_ds = train_ds
```

```
for image, label in aug_ds :
    image = augmentation(image)
```

✓ Adding BatchNormalization and Dropout

We add Dropout() and BatchNormalization() to our model architecture, which will help in improving its generalization

How do they help ?

- Dropout() : Drops out random neurons while training which forces the model to focus on every feature for prediction
- BatchNormalization() : Normalizes the inputs from previous layer for every mini-batch before sending it to the next layer. Help in maintaining the distribution of our data by making it a normal distribution (mean = 0, std. = 1)

```
model_revamp = tf.keras.Sequential([
    tf.keras.layers.Rescaling(1./255),
    #tf.keras.layers.InputLayer(input_shape = [image_size[0], image_size[1], 3]),

    tf.keras.layers.Conv2D(filters = 32, kernel_size = (3,3), padding = 'Same', activation = 'relu'),
    tf.keras.layers.Conv2D(filters = 32, kernel_size = (3,3), padding = 'Same', activation = 'relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPool2D(pool_size = (2,2)),

    tf.keras.layers.Conv2D(filters = 64, kernel_size = (3,3), padding = 'Same', activation = 'relu'),
    tf.keras.layers.Conv2D(filters = 64, kernel_size = (3,3), padding = 'Same', activation = 'relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPool2D(pool_size = (2,2)),

    tf.keras.layers.Conv2D(filters = 128, kernel_size = (3,3), padding = 'Same', activation = 'relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(rate = 0.2),

    tf.keras.layers.Dense(4, activation = 'softmax')
])

model_revamp.compile(optimizer = tf.keras.optimizers.Adam(learning_rate = 1e-4),
    loss = tf.keras.losses.CategoricalCrossentropy(),
    metrics = ['accuracy', 'Precision', 'Recall'])
```

✓ Implementing Callbacks-

We specify some callbacks to improve our model training process :

- **TensorBoard** : Enable visualizations for TensorBoard for our model's metrics, which we will plot later
- **ModelCheckpoint** : Callback to save the model weights at a given frequency, saves the best model having the lowest 'val_loss' by default
- **EarlyStopping** : Stop training when a monitored metric (val_loss) has stopped improving, after waiting for 5 epochs, specified by `patience=5`

```
log_dir_2 = "logs/Custom_CNN_revamp"

tensorboard_cb = tf.keras.callbacks.TensorBoard(log_dir = log_dir_2, histogram_freq = 1)

checkpoint_cb = tf.keras.callbacks.ModelCheckpoint("CNN_best.h5", save_best_only = True)

early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor = 'val_loss', patience = 5, restore_best_weights = True
)
```

✓ Training our Tuned model

```
history_0 = model_revamp.fit(
    train_ds,
    epochs=20,
    validation_data = valid_ds,
    callbacks=[tensorboard_cb, checkpoint_cb, early_stopping_cb])
```

```
Epoch 1/20
79/79 ━━━━━━━━━━━ 0s 164ms/step - Precision: 0.7524 - Recall: 0.4736 - accuracy: 0.6269 - loss: 0.9016WARNING:a
79/79 ━━━━━━━━━━━ 23s 206ms/step - Precision: 0.7532 - Recall: 0.4754 - accuracy: 0.6281 - loss: 0.8992 - val_P
Epoch 2/20
79/79 ━━━━━━━━━━━ 0s 132ms/step - Precision: 0.8400 - Recall: 0.7296 - accuracy: 0.7901 - loss: 0.5251WARNING:a
79/79 ━━━━━━━━━━━ 15s 169ms/step - Precision: 0.8400 - Recall: 0.7297 - accuracy: 0.7902 - loss: 0.5251 - val_P
Epoch 3/20
79/79 ━━━━━━━━━━━ 0s 128ms/step - Precision: 0.8561 - Recall: 0.7679 - accuracy: 0.8214 - loss: 0.4737WARNING:a
79/79 ━━━━━━━━━━━ 20s 165ms/step - Precision: 0.8562 - Recall: 0.7679 - accuracy: 0.8214 - loss: 0.4737 - val_P
Epoch 4/20
79/79 ━━━━━━━━━━━ 0s 129ms/step - Precision: 0.8736 - Recall: 0.7907 - accuracy: 0.8344 - loss: 0.4300WARNING:a
79/79 ━━━━━━━━━━━ 13s 166ms/step - Precision: 0.8735 - Recall: 0.7907 - accuracy: 0.8344 - loss: 0.4302 - val_P
Epoch 5/20
79/79 ━━━━━━━━━━━ 0s 133ms/step - Precision: 0.8814 - Recall: 0.8058 - accuracy: 0.8479 - loss: 0.4083WARNING:a
79/79 ━━━━━━━━━━━ 20s 154ms/step - Precision: 0.8814 - Recall: 0.8058 - accuracy: 0.8479 - loss: 0.4083 - val_P
Epoch 6/20
79/79 ━━━━━━━━━━━ 0s 135ms/step - Precision: 0.8886 - Recall: 0.8208 - accuracy: 0.8573 - loss: 0.3750WARNING:a
79/79 ━━━━━━━━━━━ 21s 155ms/step - Precision: 0.8886 - Recall: 0.8208 - accuracy: 0.8573 - loss: 0.3751 - val_P
Epoch 7/20
79/79 ━━━━━━━━━━━ 0s 136ms/step - Precision: 0.8893 - Recall: 0.8341 - accuracy: 0.8641 - loss: 0.3520WARNING:a
79/79 ━━━━━━━━━━━ 14s 172ms/step - Precision: 0.8893 - Recall: 0.8341 - accuracy: 0.8641 - loss: 0.3521 - val_P
Epoch 8/20
79/79 ━━━━━━━━━━━ 0s 130ms/step - Precision: 0.9069 - Recall: 0.8538 - accuracy: 0.8862 - loss: 0.3278WARNING:a
79/79 ━━━━━━━━━━━ 20s 166ms/step - Precision: 0.9069 - Recall: 0.8537 - accuracy: 0.8862 - loss: 0.3279 - val_P
Epoch 9/20
79/79 ━━━━━━━━━━━ 13s 166ms/step - Precision: 0.8964 - Recall: 0.8481 - accuracy: 0.8729 - loss: 0.3440 - val_P
Epoch 10/20
79/79 ━━━━━━━━━━━ 20s 153ms/step - Precision: 0.9065 - Recall: 0.8565 - accuracy: 0.8830 - loss: 0.3034 - val_P
Epoch 11/20
79/79 ━━━━━━━━━━━ 13s 168ms/step - Precision: 0.9035 - Recall: 0.8615 - accuracy: 0.8846 - loss: 0.2958 - val_P
Epoch 12/20
79/79 ━━━━━━━━━━━ 14s 170ms/step - Precision: 0.9066 - Recall: 0.8661 - accuracy: 0.8936 - loss: 0.2878 - val_P
Epoch 13/20
79/79 ━━━━━━━━━━━ 0s 133ms/step - Precision: 0.9067 - Recall: 0.8726 - accuracy: 0.8948 - loss: 0.2896WARNING:a
79/79 ━━━━━━━━━━━ 14s 170ms/step - Precision: 0.9067 - Recall: 0.8726 - accuracy: 0.8948 - loss: 0.2897 - val_P
Epoch 14/20
79/79 ━━━━━━━━━━━ 14s 172ms/step - Precision: 0.9166 - Recall: 0.8863 - accuracy: 0.9009 - loss: 0.2641 - val_P
Epoch 15/20
79/79 ━━━━━━━━━━━ 0s 132ms/step - Precision: 0.9280 - Recall: 0.8887 - accuracy: 0.9075 - loss: 0.2450WARNING:a
79/79 ━━━━━━━━━━━ 13s 168ms/step - Precision: 0.9279 - Recall: 0.8886 - accuracy: 0.9074 - loss: 0.2452 - val_P
Epoch 16/20
79/79 ━━━━━━━━━━━ 13s 168ms/step - Precision: 0.9168 - Recall: 0.8912 - accuracy: 0.9088 - loss: 0.2796 - val_P
Epoch 17/20
79/79 ━━━━━━━━━━━ 13s 168ms/step - Precision: 0.9196 - Recall: 0.8868 - accuracy: 0.9043 - loss: 0.2602 - val_P
Epoch 18/20
79/79 ━━━━━━━━━━━ 0s 136ms/step - Precision: 0.9192 - Recall: 0.8960 - accuracy: 0.9108 - loss: 0.2447WARNING:a
79/79 ━━━━━━━━━━━ 14s 172ms/step - Precision: 0.9192 - Recall: 0.8959 - accuracy: 0.9108 - loss: 0.2448 - val_P
Epoch 19/20
79/79 ━━━━━━━━━━━ 12s 157ms/step - Precision: 0.9272 - Recall: 0.9040 - accuracy: 0.9167 - loss: 0.2285 - val_P
Epoch 20/20
79/79 ━━━━━━━━━━━ 14s 171ms/step - Precision: 0.9301 - Recall: 0.9009 - accuracy: 0.9187 - loss: 0.2315 - val_P
```

```
result = model_revamp.evaluate(test_ds)
dict(zip(model_revamp.metrics_names, result))
```

```
11/11 ————— 1s 125ms/step - Precision: 0.8907 - Recall: 0.8513 - accuracy: 0.8730 - loss: 0.3327
{'loss': 0.3440038859844208, 'compile_metrics': 0.8632478713989258}
```

We observe an **accuracy jump of ~10 %** by just :

- Applying augmentation to our data
- Adding Dropout and BatchNormalization
- Implementing callbacks during training

```
l1 = []
l2 = []
l3 = []
l4 = []
```

```
conf_mat(noise_path, l1, model_revamp)
conf_mat(onion_path, l2, model_revamp)
conf_mat(potato_path, l3, model_revamp)
conf_mat(tomato_path, l4, model_revamp)
```

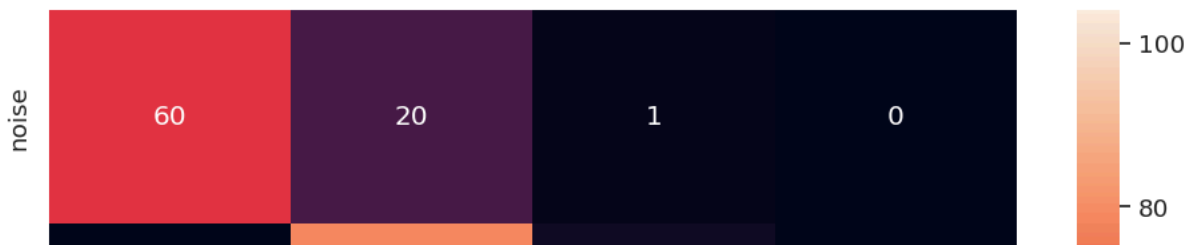
```
ax = sns.heatmap([l1, l2, l3, l4], xticklabels = class_names, yticklabels = class_names, annot = True, fmt = 'g')
ax.set(xlabel = 'Predicted label', ylabel = 'True label')
plt.show()
```

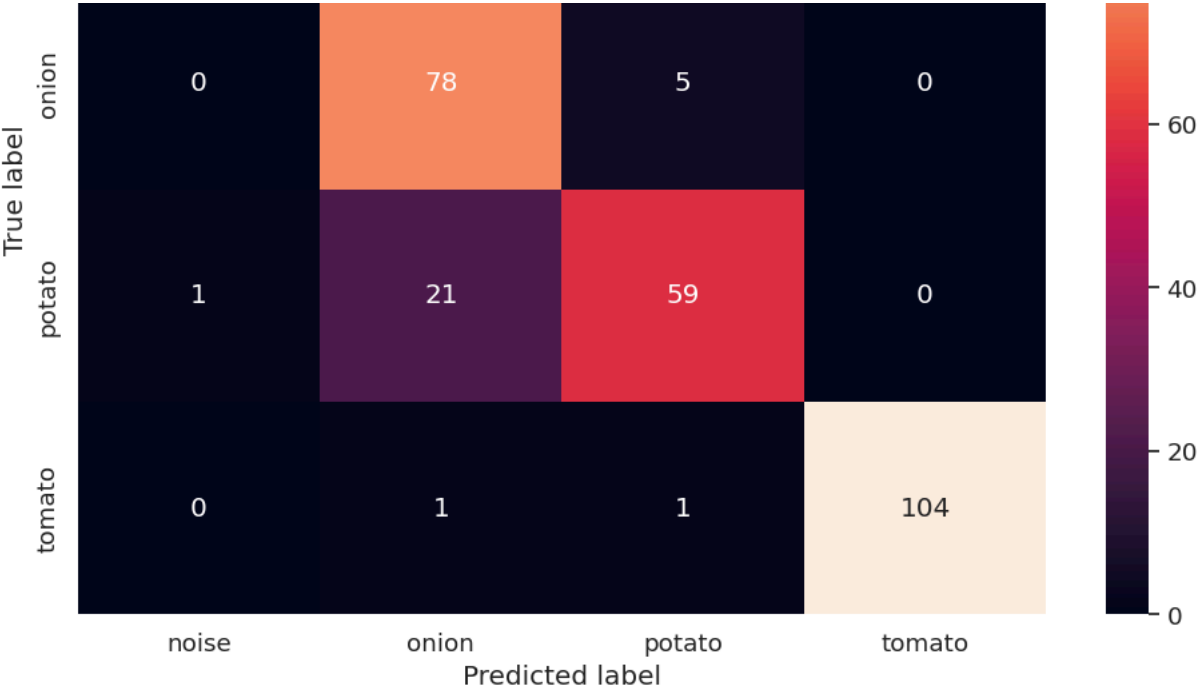
1/1 1s 662ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 29ms/step
1/1 0s 30ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 29ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 29ms/step
1/1 0s 32ms/step
1/1 0s 35ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 34ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 32ms/step
1/1 0s 30ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 30ms/step
1/1 0s 33ms/step
1/1 0s 34ms/step
1/1 0s 30ms/step
1/1 0s 34ms/step
1/1 0s 32ms/step
1/1 0s 29ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 32ms/step
1/1 0s 30ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 35ms/step
1/1 0s 33ms/step
1/1 0s 32ms/step
1/1 0s 39ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 35ms/step
1/1 0s 35ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 36ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 36ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 40ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step

1/1 ————— 0s 33ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 33ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 41ms/step
1/1 ————— 0s 34ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 35ms/step
1/1 ————— 0s 36ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 33ms/step
1/1 ————— 0s 30ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 37ms/step
1/1 ————— 0s 48ms/step
1/1 ————— 0s 46ms/step
1/1 ————— 0s 47ms/step
1/1 ————— 0s 45ms/step
1/1 ————— 0s 50ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 42ms/step
1/1 ————— 0s 59ms/step
1/1 ————— 0s 42ms/step
1/1 ————— 0s 43ms/step
1/1 ————— 0s 42ms/step
1/1 ————— 0s 50ms/step
1/1 ————— 0s 52ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 53ms/step
1/1 ————— 0s 70ms/step
1/1 ————— 0s 48ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 34ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 30ms/step
1/1 ————— 0s 33ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 33ms/step
1/1 ————— 0s 33ms/step
1/1 ————— 0s 41ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 33ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 34ms/step
1/1 ————— 0s 37ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 35ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 33ms/step
1/1 ————— 0s 34ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 33ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 34ms/step
1/1 ————— 0s 34ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 33ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 30ms/step
1/1 ————— 0s 30ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 30ms/step
1/1 ————— 0s 40ms/step
1/1 ————— 0s 34ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 35ms/step
1/1 ————— 0s 32ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 31ms/step
1/1 ————— 0s 32ms/step

1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 33ms/step
1/1 0s 34ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 34ms/step
1/1 0s 40ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 34ms/step
1/1 0s 30ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 34ms/step
1/1 0s 33ms/step
1/1 0s 32ms/step
1/1 0s 34ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 36ms/step
1/1 0s 32ms/step
1/1 0s 37ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 32ms/step
1/1 0s 43ms/step
1/1 0s 34ms/step
1/1 0s 34ms/step
1/1 0s 34ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 34ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 35ms/step
1/1 0s 58ms/step
1/1 0s 51ms/step
1/1 0s 55ms/step
1/1 0s 61ms/step
1/1 0s 49ms/step
1/1 0s 46ms/step
1/1 0s 44ms/step
1/1 0s 48ms/step
1/1 0s 50ms/step
1/1 0s 46ms/step
1/1 0s 45ms/step
1/1 0s 51ms/step
1/1 0s 51ms/step
1/1 0s 46ms/step
1/1 0s 59ms/step
1/1 0s 64ms/step
1/1 0s 51ms/step
1/1 0s 33ms/step
1/1 0s 35ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 33ms/step
1/1 0s 31ms/step
1/1 0s 31ms/step
1/1 0s 32ms/step
1/1 0s 36ms/step

1/1 _____ vs 3ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 35ms/step





In the Confusion matrix plotted above, we can observe that :

- Our model can easily differentiate between all classes with minimal error
- Potato and onion are the most confusing classes for our model
- Tomato can be easily differentiated with the smallest error

```
def plot_acc_loss(model_history, n_epochs):

    fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(15,5))
    ax = axes.ravel()

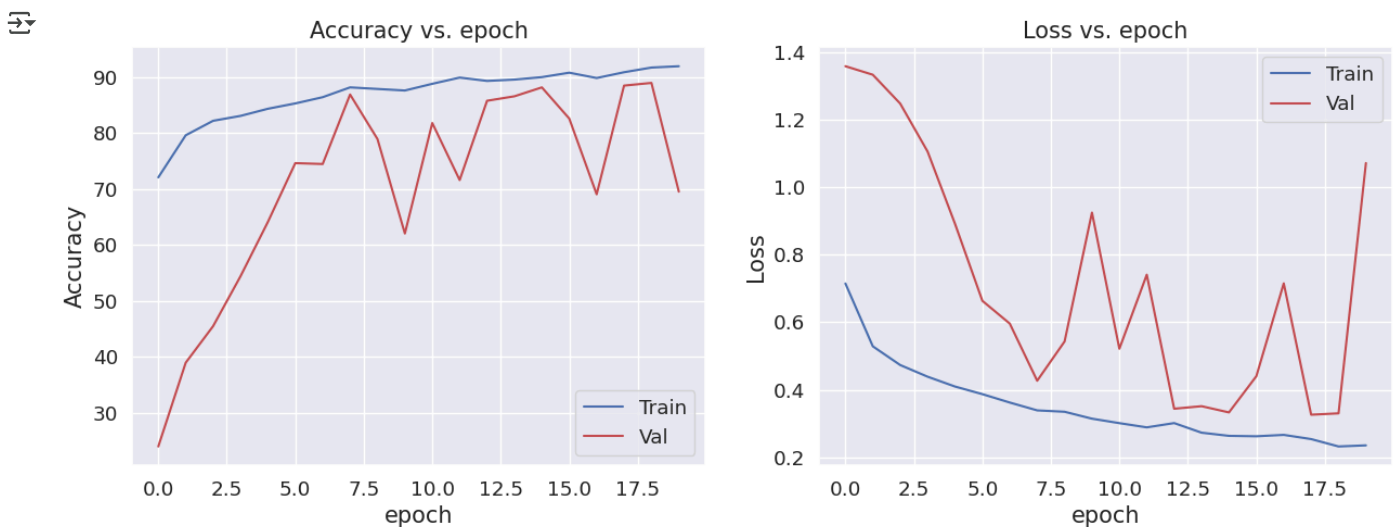
    # Get actual number of epochs from training history
    actual_epochs = len(model_history.history['accuracy'])

    #accuracy graph
    # Use actual_epochs instead of n_epochs
    ax[0].plot(range(0, actual_epochs), [acc * 100 for acc in model_history.history['accuracy']], label='Train', color='b')
    ax[0].plot(range(0, actual_epochs), [acc * 100 for acc in model_history.history['val_accuracy']], label='Val', color='r')
    ax[0].set_title('Accuracy vs. epoch', fontsize=15)
    ax[0].set_ylabel('Accuracy', fontsize=15)
    ax[0].set_xlabel('epoch', fontsize=15)
    ax[0].legend()

    #loss graph
    # Use actual_epochs instead of n_epochs
    ax[1].plot(range(0, actual_epochs), model_history.history['loss'], label='Train', color='b')
    ax[1].plot(range(0, actual_epochs), model_history.history['val_loss'], label='Val', color='r')
    ax[1].set_title('Loss vs. epoch', fontsize=15)
    ax[1].set_ylabel('Loss', fontsize=15)
    ax[1].set_xlabel('epoch', fontsize=15)
    ax[1].legend()

    #display the graph
    plt.show()
```

```
plot_acc_loss(history_0, n_epochs = 18)
```



```
classwise_accuracy(noise_path, 'noise', model_revamp)
classwise_accuracy(onion_path, 'onion', model_revamp)
classwise_accuracy(potato_path, 'potato', model_revamp)
classwise_accuracy(tomato_path, 'tomato', model_revamp)
```



```

1/1 _____ 0s 35ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 37ms/step

```

Accuracy for class tomato is 98.11% consisting of 106 images

Applying Transfer Learning:-

- Now that we solved overfitting problem of our Baseline model,
- Let's also explore and implement some state-of-the-art model architectures which can give us even higher precision and accuracy.
- We explored some of the available pretrained model which we can finetune to our data in [Keras docs](#).
We will train 3 of the most efficient and accurate pretrained models on the ImageNet dataset

The Models we will use :

- VGG-19
- ResNet
- MobileNet

✓ VGG-19

VGG-19 performance on ImageNet Dataset, let's find out how it performs on our Data after finetuning !

Model	Size (MB)	Top-1 Acc	Top-5 Acc	Parameters	Depth	Time(ms) on CPU	Time (ms) on GPU
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4

```

# load base model
base_model_3 = tf.keras.applications.vgg19.VGG19(input_shape=(256, 256, 3), include_top = False)

# base_model_3.trainable = False
# for layers in base_model_3.layers[:10]:
#     layers.trainable = False

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg19/vgg19_weights_tf_dim_ordering_t80134624/80134624 0s 0us/step

```

# append classification layer
model_3 = base_model_3.output

model_3 = tf.keras.Sequential([
    # tf.keras.layers.Lambda(
    # lambda data: tf.keras.applications.vgg19.preprocess_input( tf.cast(data, tf.float32)),
    #     input_shape=[IMG_HEIGHT, IMG_WIDTH, 3]),
    # pretrained_model,
    # Normalizing 0-255 into 0 to 1
    # tf.keras.layers.Rescaling(1./255),
    # tf.keras.layers.InputLayer(input_shape=[image_size[0], image_size[1], 3]),
    tf.keras.layers.Rescaling(1./255, input_shape=[image_size[0], image_size[1], 3]), # Define input_shape here

    base_model_3,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(rate = 0.1),
    tf.keras.layers.Dense(4, activation = 'softmax')
])

/usr/local/lib/python3.11/dist-packages/keras/src/layers/preprocessing/tf_data_layer.py:19: UserWarning: Do not pass an
super().__init__(**kwargs)

model_3.compile(optimizer = tf.keras.optimizers.Adam(learning_rate = 1e-4),
                loss = tf.keras.losses.CategoricalCrossentropy(),
                metrics = ['accuracy', 'Precision', 'Recall'])

log_dir_5 = "logs/VGG19"

tensorboard_cb = tf.keras.callbacks.TensorBoard(log_dir=log_dir_5, histogram_freq=1)

checkpoint_cb = tf.keras.callbacks.ModelCheckpoint("VGG19.keras", save_best_only=True)

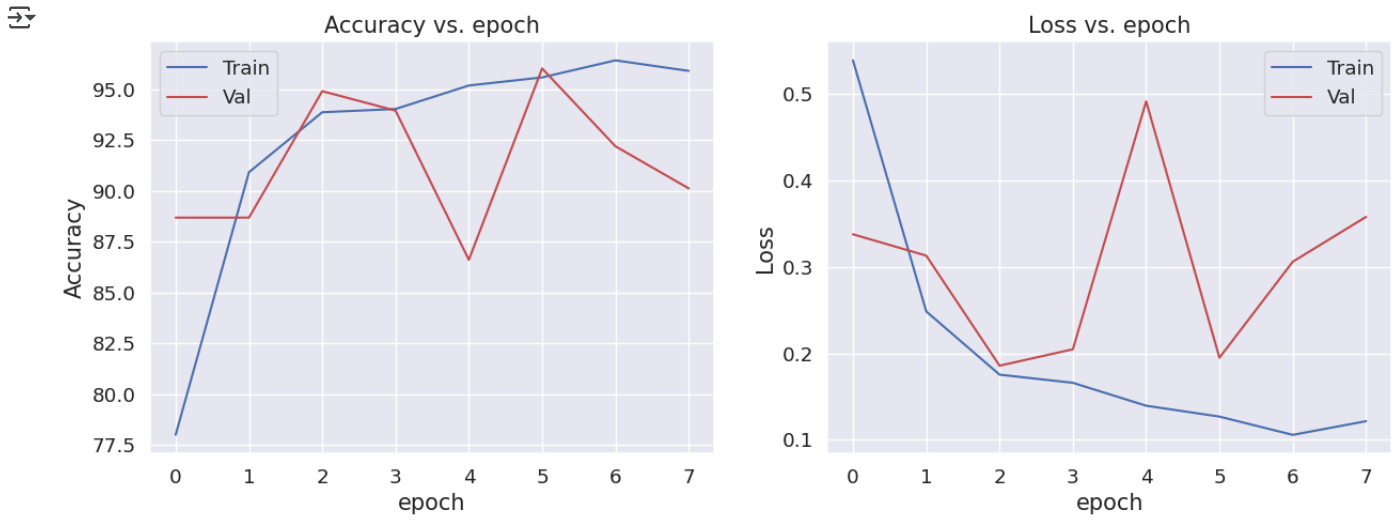
early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor = 'val_loss', patience = 5, restore_best_weights=True
)

history_3 = model_3.fit(aug_ds, validation_data = valid_ds, epochs = 20, callbacks=[tensorboard_cb, checkpoint_cb, early_stc
Epoch 1/20
79/79 ————— 179s 2s/step - Precision: 0.7281 - Recall: 0.5362 - accuracy: 0.6488 - loss: 0.8205 - val_Pre
Epoch 2/20
79/79 ————— 103s 939ms/step - Precision: 0.9023 - Recall: 0.8858 - accuracy: 0.8952 - loss: 0.2707 - val_
Epoch 3/20
79/79 ————— 74s 941ms/step - Precision: 0.9439 - Recall: 0.9250 - accuracy: 0.9303 - loss: 0.1888 - val_P
Epoch 4/20
79/79 ————— 73s 926ms/step - Precision: 0.9483 - Recall: 0.9376 - accuracy: 0.9423 - loss: 0.1614 - val_P
Epoch 5/20
79/79 ————— 69s 870ms/step - Precision: 0.9575 - Recall: 0.9481 - accuracy: 0.9528 - loss: 0.1292 - val_P
Epoch 6/20
79/79 ————— 73s 923ms/step - Precision: 0.9508 - Recall: 0.9418 - accuracy: 0.9468 - loss: 0.1582 - val_P
Epoch 7/20
79/79 ————— 83s 938ms/step - Precision: 0.9793 - Recall: 0.9766 - accuracy: 0.9786 - loss: 0.0745 - val_P
Epoch 8/20
79/79 ————— 81s 920ms/step - Precision: 0.9608 - Recall: 0.9555 - accuracy: 0.9589 - loss: 0.1209 - val_P

```

✓ Evaluating VGG-19

```
plot_acc_loss(history_3, n_epochs = 16)
```



```
classwise_accuracy(noise_path, 'noise', model_3)
classwise_accuracy(onion_path, 'onion', model_3)
classwise_accuracy(potato_path, 'potato', model_3)
classwise_accuracy(tomato_path, 'tomato', model_3)
```

```
1/1 ===== 0s 4ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 50ms/step
```

Accuracy for class tomato is 100.0% consisting of 106 images

```
l1 = []
l2 = []
l3 = []
l4 = []
```

```
conf_mat(noise_path, l1, model_3)
conf_mat(onion_path, l2, model_3)
conf_mat(potato_path, l3, model_3)
conf_mat(tomato_path, l4, model_3)
```

```
ax = sns.heatmap([l1, l2, l3, l4], xticklabels=class_names, yticklabels=class_names, annot=True, fmt='g')
ax.set(xlabel='Predicted label', ylabel='True label')
plt.show()
```

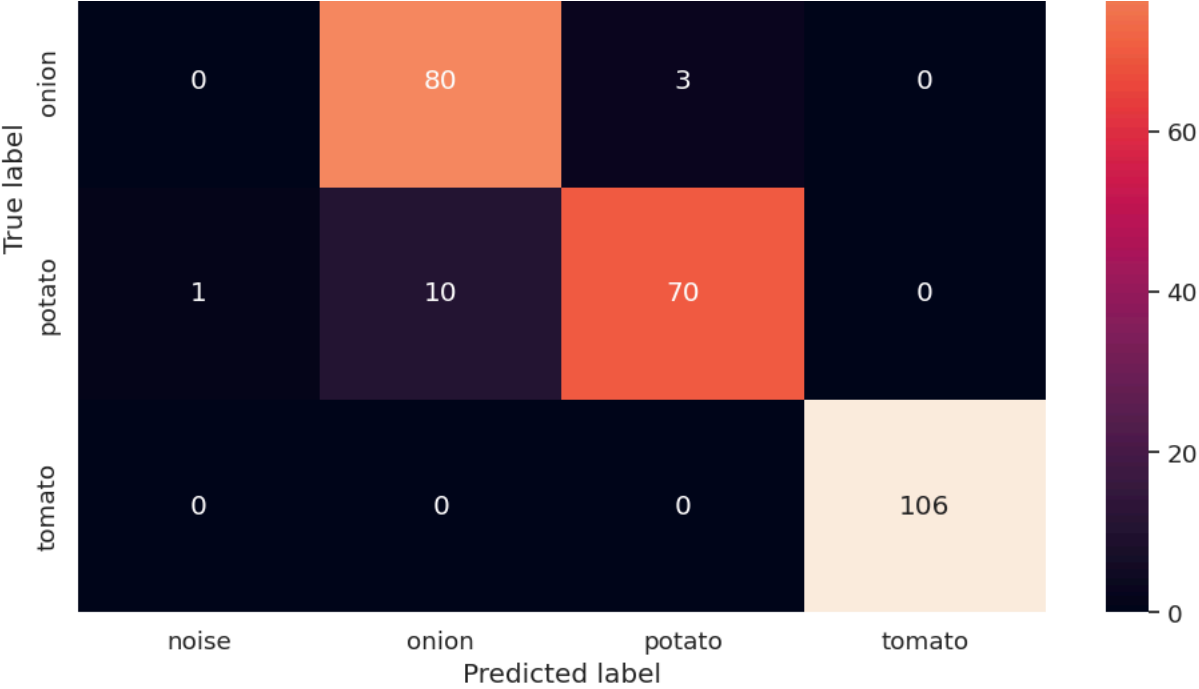
1/1 0s 50ms/step
1/1 0s 49ms/step
1/1 0s 48ms/step
1/1 0s 46ms/step
1/1 0s 46ms/step
1/1 0s 47ms/step
1/1 0s 45ms/step
1/1 0s 45ms/step
1/1 0s 47ms/step
1/1 0s 46ms/step
1/1 0s 50ms/step
1/1 0s 51ms/step
1/1 0s 47ms/step
1/1 0s 47ms/step
1/1 0s 57ms/step
1/1 0s 47ms/step
1/1 0s 48ms/step
1/1 0s 46ms/step
1/1 0s 45ms/step
1/1 0s 47ms/step
1/1 0s 46ms/step
1/1 0s 45ms/step
1/1 0s 57ms/step
1/1 0s 48ms/step
1/1 0s 47ms/step
1/1 0s 47ms/step
1/1 0s 47ms/step
1/1 0s 47ms/step
1/1 0s 47ms/step
1/1 0s 53ms/step
1/1 0s 61ms/step
1/1 0s 61ms/step
1/1 0s 56ms/step
1/1 0s 58ms/step
1/1 0s 62ms/step
1/1 0s 60ms/step
1/1 0s 57ms/step
1/1 0s 58ms/step
1/1 0s 62ms/step
1/1 0s 60ms/step
1/1 0s 64ms/step
1/1 0s 58ms/step
1/1 0s 60ms/step
1/1 0s 66ms/step
1/1 0s 58ms/step
1/1 0s 63ms/step
1/1 0s 63ms/step
1/1 0s 65ms/step
1/1 0s 47ms/step
1/1 0s 46ms/step
1/1 0s 48ms/step
1/1 0s 46ms/step
1/1 0s 46ms/step
1/1 0s 51ms/step
1/1 0s 47ms/step
1/1 0s 45ms/step
1/1 0s 46ms/step
1/1 0s 50ms/step
1/1 0s 47ms/step
1/1 0s 47ms/step
1/1 0s 47ms/step
1/1 0s 47ms/step
1/1 0s 48ms/step
1/1 0s 46ms/step
1/1 0s 47ms/step
1/1 0s 46ms/step
1/1 0s 45ms/step
1/1 0s 51ms/step
1/1 0s 45ms/step
1/1 0s 47ms/step
1/1 0s 46ms/step
1/1 0s 49ms/step
1/1 0s 47ms/step
1/1 0s 48ms/step
1/1 0s 48ms/step
1/1 0s 45ms/step
1/1 0s 45ms/step
1/1 0s 48ms/step
1/1 0s 49ms/step
1/1 0s 46ms/step
1/1 0s 46ms/step
1/1 0s 48ms/step
1/1 0s 51ms/step
1/1 0s 51ms/step
1/1 0s 50ms/step
1/1 0s 49ms/step
1/1 0s 49ms/step
1/1 0s 56ms/step
1/1 0s 49ms/step

1/1 _____ 0s 48ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 57ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 54ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 61ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 66ms/step
1/1 _____ 0s 64ms/step
1/1 _____ 0s 64ms/step
1/1 _____ 0s 62ms/step
1/1 _____ 0s 62ms/step
1/1 _____ 0s 61ms/step
1/1 _____ 0s 63ms/step
1/1 _____ 0s 61ms/step
1/1 _____ 0s 58ms/step
1/1 _____ 0s 66ms/step
1/1 _____ 0s 71ms/step
1/1 _____ 0s 61ms/step
1/1 _____ 0s 62ms/step
1/1 _____ 0s 64ms/step
1/1 _____ 0s 63ms/step
1/1 _____ 0s 64ms/step
1/1 _____ 0s 59ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 48ms/step

1/1 _____ 0s 54ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 54ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 56ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 57ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 58ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 58ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 68ms/step
1/1 _____ 0s 63ms/step
1/1 _____ 0s 61ms/step
1/1 _____ 0s 63ms/step
1/1 _____ 0s 63ms/step
1/1 _____ 0s 61ms/step
1/1 _____ 0s 59ms/step
1/1 _____ 0s 60ms/step
1/1 _____ 0s 60ms/step
1/1 _____ 0s 60ms/step
1/1 _____ 0s 62ms/step
1/1 _____ 0s 64ms/step
1/1 _____ 0s 65ms/step
1/1 _____ 0s 66ms/step
1/1 _____ 0s 68ms/step

```
1/1 _____ 0s 69ms/step
1/1 _____ 0s 66ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 62ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 58ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 60ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 57ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 58ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 56ms/step
1/1 _____ 0s 50ms/step
```





```
result = model_3.evaluate(test_ds)
dict(zip(model_3.metrics_names, result))
```

```
11/11 ————— 17s 2s/step - Precision: 0.9029 - Recall: 0.8891 - accuracy: 0.8964 - loss: 0.3248
{'loss': 0.3362331986427307, 'compile_metrics': 0.8860399127006531}
```

✓ ResNet Model

ResNet performance on ImageNet Dataset, let's find out how it performs on our Data !

Model	Size (MB)	Top-1 Acc	Top-5 Acc	Parameters	Depth	Time(ms) on CPU	Time (ms) on GPU
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6

```
# load base model
base_model_1 = tf.keras.applications.resnet50.ResNet50(input_shape=(256, 256, 3), include_top = False)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_orderi
94765736/94765736 ————— 1s 0us/step
```

```
# append classification layer
model_1 = base_model_1.output

model_1 = tf.keras.Sequential([
    #Normalizing 0-255 into 0 to 1
    # tf.keras.layers.Rescaling(1./255),
    # tf.keras.layers.InputLayer(input_shape=[image_size[0], image_size[1], 3]),
    tf.keras.layers.Rescaling(1./255, input_shape=[image_size[0], image_size[1], 3]), # Define input_shape here
    base_model_1,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(rate = 0.1),
    tf.keras.layers.Dense(4, activation = 'softmax')
])
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/preprocessing/tf_data_layer.py:19: UserWarning: Do not pass an
super().__init__(**kwargs)
```

```
model_1.compile(optimizer = tf.keras.optimizers.Adam(learning_rate = 1e-4),
                loss = tf.keras.losses.CategoricalCrossentropy(),
                metrics = ['accuracy', 'Precision', 'Recall'])
```

```
log_dir_3 = "logs/ResNet"
```

```
tensorboard_cb = tf.keras.callbacks.TensorBoard(log_dir=log_dir_3, histogram_freq=1)
```

```
checkpoint_cb = tf.keras.callbacks.ModelCheckpoint("ResNet.h5", save_best_only=True)
```

```
early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor = 'val_loss', patience = 5, restore_best_weights=True
)
```

```
history_1 = model_1.fit(aug_ds, validation_data = valid_ds, epochs = 20, callbacks=[tensorboard_cb, checkpoint_cb, early_stc
```

```
Epoch 1/20
79/79 ————— 0s 669ms/step - Precision: 0.8662 - Recall: 0.8112 - accuracy: 0.8442 - loss: 0.4312WARNING:a
79/79 ————— 136s 918ms/step - Precision: 0.8671 - Recall: 0.8125 - accuracy: 0.8452 - loss: 0.4282 - val_
Epoch 2/20
79/79 ————— 39s 488ms/step - Precision: 0.9955 - Recall: 0.9945 - accuracy: 0.9955 - loss: 0.0177 - val_P
Epoch 3/20
79/79 ————— 38s 476ms/step - Precision: 0.9939 - Recall: 0.9939 - accuracy: 0.9939 - loss: 0.0211 - val_P
Epoch 4/20
79/79 ————— 41s 473ms/step - Precision: 0.9920 - Recall: 0.9908 - accuracy: 0.9920 - loss: 0.0304 - val_P
Epoch 5/20
79/79 ————— 0s 354ms/step - Precision: 0.9978 - Recall: 0.9977 - accuracy: 0.9977 - loss: 0.0071WARNING:a
79/79 ————— 39s 494ms/step - Precision: 0.9978 - Recall: 0.9977 - accuracy: 0.9977 - loss: 0.0072 - val_P
Epoch 6/20
79/79 ————— 39s 470ms/step - Precision: 0.9925 - Recall: 0.9920 - accuracy: 0.9925 - loss: 0.0244 - val_P
Epoch 7/20
79/79 ————— 0s 355ms/step - Precision: 0.9944 - Recall: 0.9942 - accuracy: 0.9944 - loss: 0.0153WARNING:a
79/79 ————— 39s 498ms/step - Precision: 0.9945 - Recall: 0.9942 - accuracy: 0.9945 - loss: 0.0152 - val_P
Epoch 8/20
79/79 ————— 0s 350ms/step - Precision: 0.9998 - Recall: 0.9998 - accuracy: 0.9998 - loss: 0.0025WARNING:a
79/79 ————— 41s 498ms/step - Precision: 0.9998 - Recall: 0.9998 - accuracy: 0.9998 - loss: 0.0025 - val_P
Epoch 9/20
79/79 ————— 0s 355ms/step - Precision: 0.9978 - Recall: 0.9978 - accuracy: 0.9978 - loss: 0.0069WARNING:a
79/79 ————— 41s 497ms/step - Precision: 0.9978 - Recall: 0.9978 - accuracy: 0.9978 - loss: 0.0070 - val_P
Epoch 10/20
79/79 ————— 0s 351ms/step - Precision: 0.9871 - Recall: 0.9865 - accuracy: 0.9868 - loss: 0.0345WARNING:a
79/79 ————— 40s 481ms/step - Precision: 0.9871 - Recall: 0.9865 - accuracy: 0.9868 - loss: 0.0345 - val_P
```

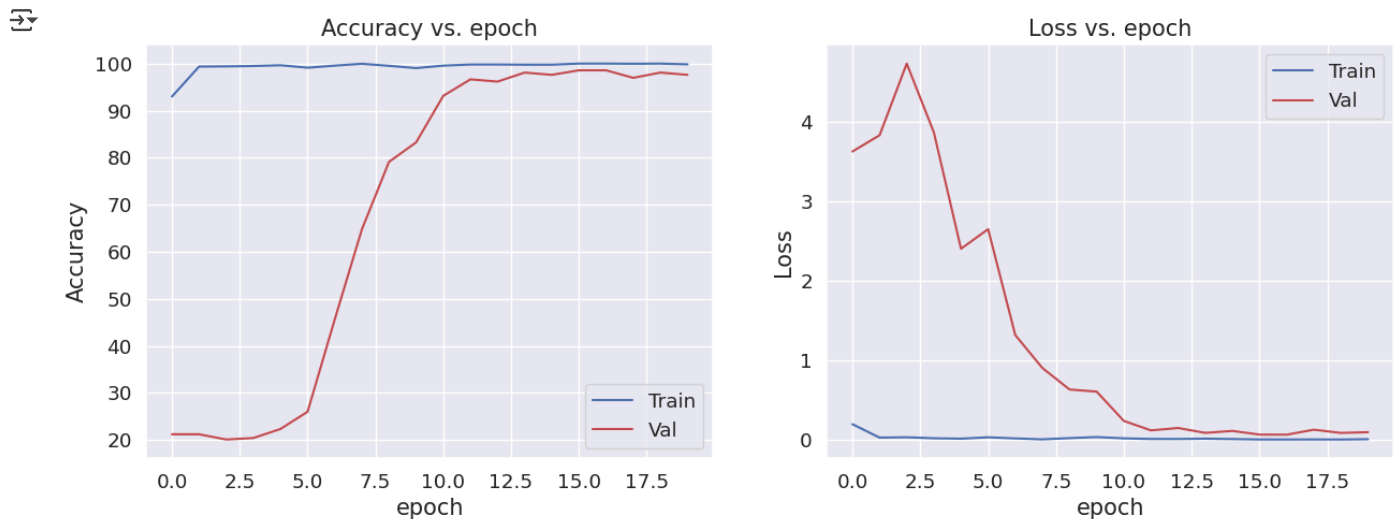
```

Epoch 11/20
79/79 0s 353ms/step - Precision: 0.9930 - Recall: 0.9924 - accuracy: 0.9927 - loss: 0.0206WARNING:a
79/79 42s 493ms/step - Precision: 0.9931 - Recall: 0.9925 - accuracy: 0.9928 - loss: 0.0205 - val_P
Epoch 12/20
79/79 0s 352ms/step - Precision: 0.9979 - Recall: 0.9979 - accuracy: 0.9979 - loss: 0.0084WARNING:a
79/79 39s 494ms/step - Precision: 0.9979 - Recall: 0.9979 - accuracy: 0.9979 - loss: 0.0083 - val_P
Epoch 13/20
79/79 39s 469ms/step - Precision: 0.9986 - Recall: 0.9986 - accuracy: 0.9986 - loss: 0.0060 - val_P
Epoch 14/20
79/79 0s 354ms/step - Precision: 0.9985 - Recall: 0.9982 - accuracy: 0.9982 - loss: 0.0104WARNING:a
79/79 43s 497ms/step - Precision: 0.9985 - Recall: 0.9982 - accuracy: 0.9982 - loss: 0.0104 - val_P
Epoch 15/20
79/79 38s 482ms/step - Precision: 0.9979 - Recall: 0.9979 - accuracy: 0.9979 - loss: 0.0056 - val_P
Epoch 16/20
79/79 0s 353ms/step - Precision: 1.0000 - Recall: 1.0000 - accuracy: 1.0000 - loss: 0.0017WARNING:a
79/79 42s 495ms/step - Precision: 1.0000 - Recall: 1.0000 - accuracy: 1.0000 - loss: 0.0017 - val_P
Epoch 17/20
79/79 0s 351ms/step - Precision: 1.0000 - Recall: 1.0000 - accuracy: 1.0000 - loss: 0.0012WARNING:a
79/79 38s 483ms/step - Precision: 1.0000 - Recall: 1.0000 - accuracy: 1.0000 - loss: 0.0012 - val_P
Epoch 18/20
79/79 40s 501ms/step - Precision: 0.9999 - Recall: 0.9999 - accuracy: 0.9999 - loss: 9.0977e-04 - v
Epoch 19/20
79/79 38s 461ms/step - Precision: 1.0000 - Recall: 1.0000 - accuracy: 1.0000 - loss: 8.1031e-04 - v
Epoch 20/20
79/79 36s 461ms/step - Precision: 0.9991 - Recall: 0.9991 - accuracy: 0.9991 - loss: 0.0023 - val_P

```

✓ Evaluating ResNet

```
plot_acc_loss(history_1, n_epochs = 8)
```



```

classwise_accuracy(noise_path, 'noise', model_1)
classwise_accuracy(onion_path, 'onion', model_1)
classwise_accuracy(potato_path, 'potato', model_1)
classwise_accuracy(tomato_path, 'tomato', model_1)

```



```

1/1 ===== 0s 44ms/step
1/1 ===== 0s 43ms/step
1/1 ===== 0s 46ms/step
1/1 ===== 0s 43ms/step
1/1 ===== 0s 44ms/step
1/1 ===== 0s 51ms/step
1/1 ===== 0s 45ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 45ms/step
1/1 ===== 0s 46ms/step
1/1 ===== 0s 45ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 45ms/step
1/1 ===== 0s 45ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 46ms/step
1/1 ===== 0s 46ms/step
1/1 ===== 0s 46ms/step
1/1 ===== 0s 51ms/step
1/1 ===== 0s 46ms/step
1/1 ===== 0s 46ms/step
1/1 ===== 0s 45ms/step
1/1 ===== 0s 44ms/step
1/1 ===== 0s 44ms/step
1/1 ===== 0s 45ms/step
1/1 ===== 0s 51ms/step
1/1 ===== 0s 44ms/step
1/1 ===== 0s 51ms/step
1/1 ===== 0s 46ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 55ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 45ms/step

```

Accuracy for class tomato is 100.0% consisting of 106 images

```

l1 = []
l2 = []
l3 = []
l4 = []

```

```

conf_mat(noise_path, l1, model_1)
conf_mat(onion_path, l2, model_1)
conf_mat(potato_path, l3, model_1)
conf_mat(tomato_path, l4, model_1)

```

```

ax = sns.heatmap([l1, l2, l3, l4], xticklabels=class_names, yticklabels=class_names, annot=True, fmt='g')
ax.set(xlabel='Predicted label', ylabel='True label')
plt.show()

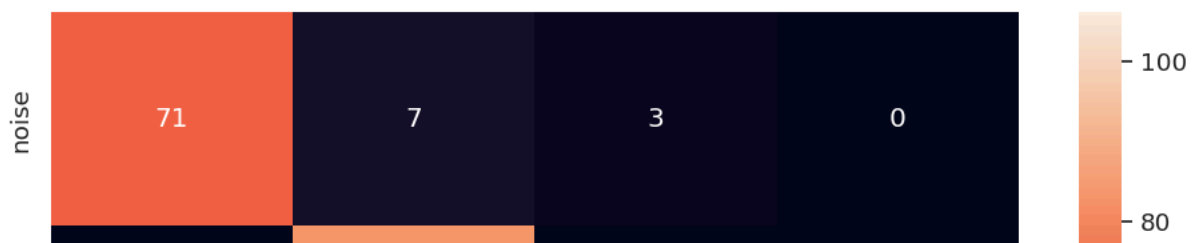
```

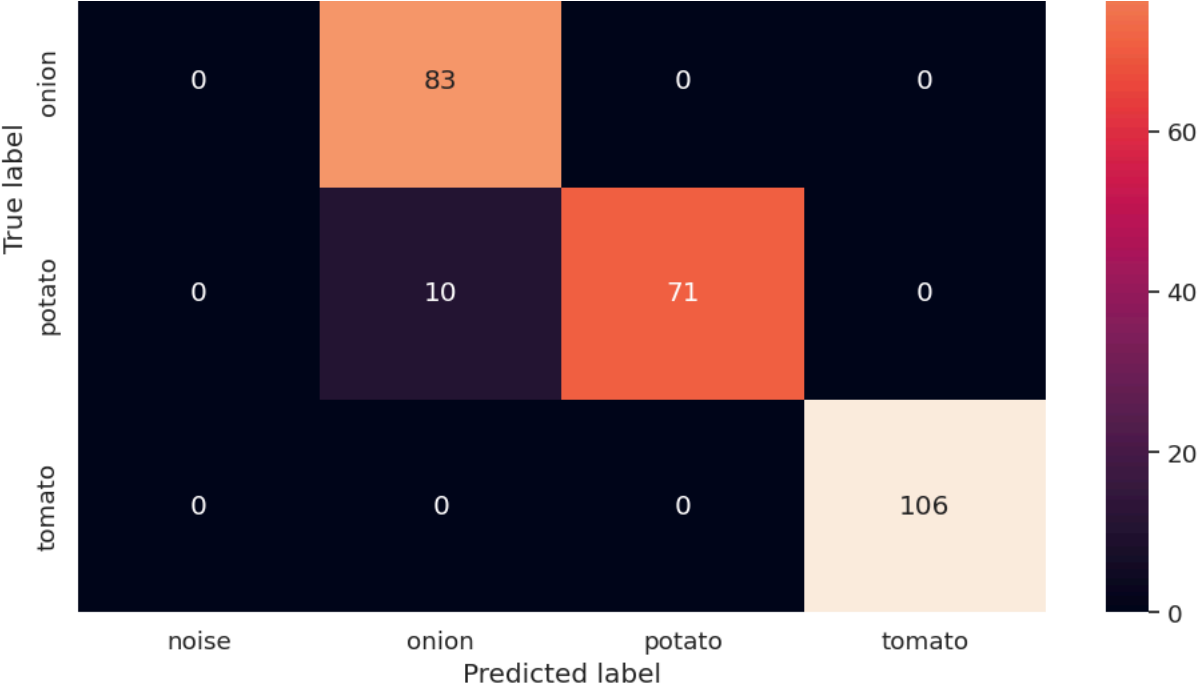
1/1 0s 51ms/step
1/1 0s 42ms/step
1/1 0s 44ms/step
1/1 0s 42ms/step
1/1 0s 41ms/step
1/1 0s 44ms/step
1/1 0s 44ms/step
1/1 0s 44ms/step
1/1 0s 44ms/step
1/1 0s 46ms/step
1/1 0s 48ms/step
1/1 0s 42ms/step
1/1 0s 44ms/step
1/1 0s 40ms/step
1/1 0s 45ms/step
1/1 0s 45ms/step
1/1 0s 41ms/step
1/1 0s 41ms/step
1/1 0s 42ms/step
1/1 0s 43ms/step
1/1 0s 45ms/step
1/1 0s 43ms/step
1/1 0s 43ms/step
1/1 0s 44ms/step
1/1 0s 42ms/step
1/1 0s 41ms/step
1/1 0s 44ms/step
1/1 0s 41ms/step
1/1 0s 40ms/step
1/1 0s 41ms/step
1/1 0s 44ms/step
1/1 0s 46ms/step
1/1 0s 43ms/step
1/1 0s 47ms/step
1/1 0s 43ms/step
1/1 0s 43ms/step
1/1 0s 43ms/step
1/1 0s 42ms/step
1/1 0s 40ms/step
1/1 0s 43ms/step
1/1 0s 41ms/step
1/1 0s 41ms/step
1/1 0s 45ms/step
1/1 0s 44ms/step
1/1 0s 43ms/step
1/1 0s 48ms/step
1/1 0s 43ms/step
1/1 0s 41ms/step
1/1 0s 41ms/step
1/1 0s 42ms/step
1/1 0s 43ms/step
1/1 0s 40ms/step
1/1 0s 42ms/step
1/1 0s 51ms/step
1/1 0s 44ms/step
1/1 0s 46ms/step
1/1 0s 44ms/step
1/1 0s 45ms/step
1/1 0s 48ms/step
1/1 0s 43ms/step
1/1 0s 43ms/step
1/1 0s 43ms/step
1/1 0s 49ms/step
1/1 0s 42ms/step
1/1 0s 42ms/step
1/1 0s 41ms/step
1/1 0s 43ms/step
1/1 0s 50ms/step
1/1 0s 45ms/step
1/1 0s 42ms/step
1/1 0s 46ms/step
1/1 0s 42ms/step
1/1 0s 43ms/step
1/1 0s 45ms/step
1/1 0s 43ms/step
1/1 0s 44ms/step
1/1 0s 42ms/step
1/1 0s 40ms/step
1/1 0s 47ms/step
1/1 0s 47ms/step
1/1 0s 43ms/step
1/1 0s 45ms/step
1/1 0s 44ms/step
1/1 0s 44ms/step
1/1 0s 44ms/step
1/1 0s 45ms/step
1/1 0s 43ms/step
1/1 0s 58ms/step
1/1 0s 63ms/step
1/1 0s 56ms/step
1/1 0s 56ms/step

1/1 _____ 0s 61ms/step
1/1 _____ 0s 57ms/step
1/1 _____ 0s 56ms/step
1/1 _____ 0s 54ms/step
1/1 _____ 0s 62ms/step
1/1 _____ 0s 61ms/step
1/1 _____ 0s 57ms/step
1/1 _____ 0s 56ms/step
1/1 _____ 0s 56ms/step
1/1 _____ 0s 59ms/step
1/1 _____ 0s 65ms/step
1/1 _____ 0s 56ms/step
1/1 _____ 0s 61ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 54ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 44ms/step

1/1 _____ 0s 43ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 55ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 56ms/step
1/1 _____ 0s 72ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 56ms/step
1/1 _____ 0s 55ms/step
1/1 _____ 0s 66ms/step
1/1 _____ 0s 55ms/step
1/1 _____ 0s 56ms/step
1/1 _____ 0s 67ms/step
1/1 _____ 0s 62ms/step
1/1 _____ 0s 59ms/step
1/1 _____ 0s 57ms/step
1/1 _____ 0s 65ms/step
1/1 _____ 0s 60ms/step
1/1 _____ 0s 61ms/step
1/1 _____ 0s 60ms/step
1/1 _____ 0s 69ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 54ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 44ms/step

```
1/1 _____ 0s 45ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 62ms/step
1/1 _____ 0s 59ms/step
1/1 _____ 0s 59ms/step
1/1 _____ 0s 54ms/step
1/1 _____ 0s 54ms/step
1/1 _____ 0s 58ms/step
1/1 _____ 0s 54ms/step
1/1 _____ 0s 56ms/step
1/1 _____ 0s 57ms/step
1/1 _____ 0s 64ms/step
1/1 _____ 0s 72ms/step
1/1 _____ 0s 60ms/step
1/1 _____ 0s 61ms/step
1/1 _____ 0s 64ms/step
1/1 _____ 0s 56ms/step
1/1 _____ 0s 58ms/step
1/1 _____ 0s 72ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 43ms/step
```





```
result = model_1.evaluate(test_ds)
dict(zip(model_1.metrics_names, result))
```

We achieved a test accuracy of **94%** on our finetuned ResNet Model, let's finetune a smaller sized model to deploy on our Mobile App !

✓ MobileNet Model

MobileNet performance on ImageNet Dataset, let's find out how it performs on our Data !

Model	Size (MB)	Top-1 Acc	Top-5 Acc	Parameters	Depth	Time(ms) on CPU	Time (ms) on GPU
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4

```
# load base model
base_model_2 = tf.keras.applications.mobilenet.MobileNet(input_shape=(256, 256, 3), include_top = False)

<ipython-input-76-5b9d90e375d2>:2: UserWarning: `input_shape` is undefined or non-square, or `rows` is not in [128, 160,
base_model_2 = tf.keras.applications.mobilenet.MobileNet(input_shape=(256, 256, 3), include_top = False)
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet/mobilenet_1.0.224_tf_no_top
17225924/17225924 ————— 0s 0us/step
```

```
# append classification layer
model_2 = base_model_2.output
```

```
model_2 = tf.keras.Sequential([
    #Normalizing 0-255 into 0 to 1
    # tf.keras.layers.Rescaling(1./255),
    # tf.keras.layers.InputLayer(input_shape=[image_size[0], image_size[1], 3]),
    tf.keras.layers.Rescaling(1./255, input_shape=[image_size[0], image_size[1], 3]),
    base_model_2,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(rate = 0.1),
    tf.keras.layers.Dense(4, activation = 'softmax')
])
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/preprocessing/tf_data_layer.py:19: UserWarning: Do not pass an
super().__init__(**kwargs)
```

```
log_dir_4 = "logs/MobileNet"
```

```
tensorboard_cb = tf.keras.callbacks.TensorBoard(log_dir=log_dir_4, histogram_freq=1)
```

```
checkpoint_cb = tf.keras.callbacks.ModelCheckpoint("MobileNet.h5", save_best_only=True)
```

```
early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor = 'val_loss', patience = 5, restore_best_weights=True
)
```

```
model_2.compile(optimizer = tf.keras.optimizers.Adam(learning_rate = 1e-4),
    loss = tf.keras.losses.CategoricalCrossentropy(),
    metrics = ['accuracy', 'Precision', 'Recall'])
```

```
history_2 = model_2.fit(aug_ds, validation_data = valid_ds, epochs = 20, callbacks=[tensorboard_cb, checkpoint_cb, early_stc
```

```
Epoch 1/20
79/79 ————— 0s 252ms/step - Precision: 0.8046 - Recall: 0.7457 - accuracy: 0.7742 - loss: 0.6107WARNING:a
79/79 ————— 63s 419ms/step - Precision: 0.8060 - Recall: 0.7474 - accuracy: 0.7757 - loss: 0.6068 - val_P
Epoch 2/20
78/79 ————— 0s 117ms/step - Precision: 0.9918 - Recall: 0.9908 - accuracy: 0.9918 - loss: 0.0337WARNING:a
79/79 ————— 47s 173ms/step - Precision: 0.9918 - Recall: 0.9908 - accuracy: 0.9918 - loss: 0.0336 - val_P
Epoch 3/20
78/79 ————— 0s 116ms/step - Precision: 0.9975 - Recall: 0.9972 - accuracy: 0.9975 - loss: 0.0151WARNING:a
79/79 ————— 20s 173ms/step - Precision: 0.9975 - Recall: 0.9972 - accuracy: 0.9975 - loss: 0.0151 - val_P
Epoch 4/20
78/79 ————— 0s 116ms/step - Precision: 0.9982 - Recall: 0.9979 - accuracy: 0.9982 - loss: 0.0100WARNING:a
79/79 ————— 14s 171ms/step - Precision: 0.9982 - Recall: 0.9979 - accuracy: 0.9982 - loss: 0.0100 - val_P
Epoch 5/20
79/79 ————— 0s 111ms/step - Precision: 0.9990 - Recall: 0.9990 - accuracy: 0.9990 - loss: 0.0081WARNING:a
79/79 ————— 20s 169ms/step - Precision: 0.9990 - Recall: 0.9990 - accuracy: 0.9990 - loss: 0.0081 - val_P
Epoch 6/20
78/79 ————— 0s 110ms/step - Precision: 0.9999 - Recall: 0.9996 - accuracy: 0.9996 - loss: 0.0059WARNING:a
79/79 ————— 13s 160ms/step - Precision: 0.9999 - Recall: 0.9996 - accuracy: 0.9996 - loss: 0.0059 - val_P
Epoch 7/20
79/79 ————— 12s 156ms/step - Precision: 0.9999 - Recall: 0.9999 - accuracy: 0.9999 - loss: 0.0024 - val_P
Epoch 8/20
79/79 ————— 21s 157ms/step - Precision: 0.9981 - Recall: 0.9980 - accuracy: 0.9980 - loss: 0.0064 - val_P
Epoch 9/20
79/79 ————— 21s 169ms/step - Precision: 0.9996 - Recall: 0.9996 - accuracy: 0.9996 - loss: 0.0036 - val_P
```

Epoch 10/20

79/79 — 12s 153ms/step - Precision: 1.0000 - Recall: 0.9993 - accuracy: 0.9996 - loss: 0.0028 - val_P

Epoch 11/20

79/79 — 12s 153ms/step - Precision: 1.0000 - Recall: 1.0000 - accuracy: 1.0000 - loss: 0.0016 - val_P

model_2.summary()

Model: "sequential_8"

Layer (type)	Output Shape	Param #
rescaling_7 (Rescaling)	(None, 256, 256, 3)	0
mobilenet_1.00_224 (Functional)	(None, 8, 8, 1024)	3,228,864
global_average_pooling2d_7 (GlobalAveragePooling2D)	(None, 1024)	0
dropout_6 (Dropout)	(None, 1024)	0
dense_7 (Dense)	(None, 4)	4,100

Total params: 9,655,118 (36.83 MB)

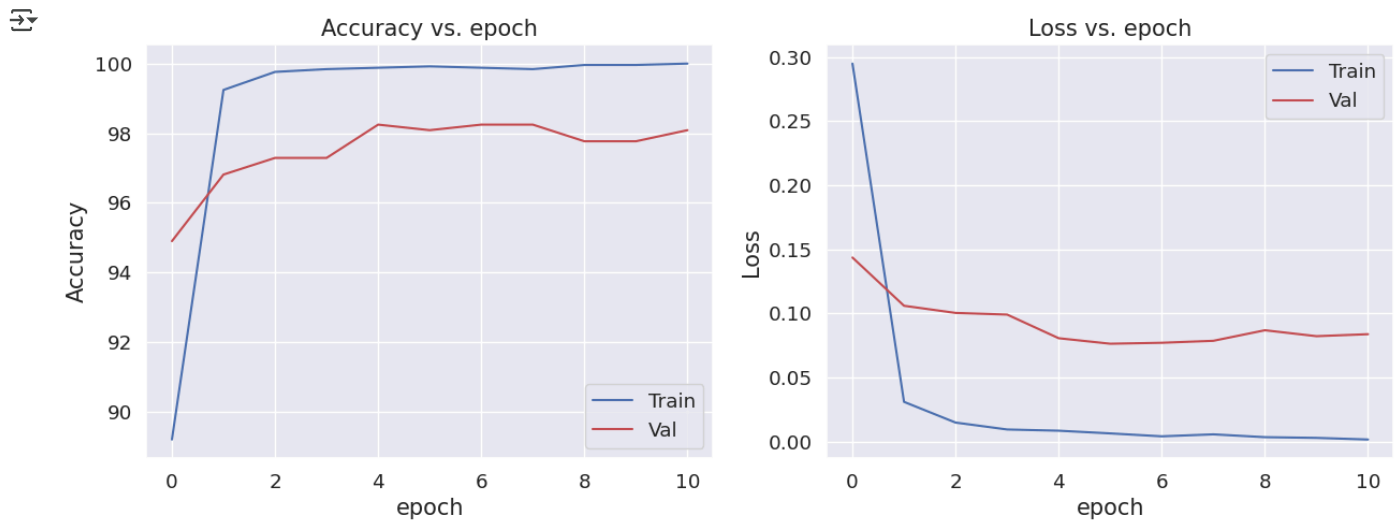
Trainable params: 3,211,076 (12.25 MB)

Non-trainable params: 21,888 (85.50 KB)

Optimizer params: 6,422,154 (24.50 MB)

✓ Evaluating MobileNet

plot_acc_loss(history_2, n_epochs = 16)



```
classwise_accuracy(noise_path, 'noise', model_2)
classwise_accuracy(onion_path, 'onion', model_2)
classwise_accuracy(potato_path, 'potato', model_2)
classwise_accuracy(tomato_path, 'tomato', model_2)
```



```

1/1 ===== 0s 33ms/step
1/1 ===== 0s 37ms/step
1/1 ===== 0s 37ms/step
1/1 ===== 0s 34ms/step
1/1 ===== 0s 34ms/step
1/1 ===== 0s 36ms/step
1/1 ===== 0s 33ms/step
1/1 ===== 0s 33ms/step
1/1 ===== 0s 33ms/step
1/1 ===== 0s 33ms/step
1/1 ===== 0s 34ms/step
1/1 ===== 0s 38ms/step
1/1 ===== 0s 37ms/step
1/1 ===== 0s 34ms/step
1/1 ===== 0s 37ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 51ms/step
1/1 ===== 0s 45ms/step
1/1 ===== 0s 55ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 53ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 57ms/step
1/1 ===== 0s 63ms/step
1/1 ===== 0s 50ms/step
1/1 ===== 0s 55ms/step
1/1 ===== 0s 44ms/step
1/1 ===== 0s 50ms/step
1/1 ===== 0s 56ms/step
1/1 ===== 0s 52ms/step
1/1 ===== 0s 33ms/step
1/1 ===== 0s 33ms/step
1/1 ===== 0s 37ms/step

```

Accuracy for class tomato is 100.0% consisting of 106 images

```

l1 = []
l2 = []
l3 = []
l4 = []

```

```

conf_mat(noise_path, l1, model_2)
conf_mat(onion_path, l2, model_2)
conf_mat(potato_path, l3, model_2)
conf_mat(tomato_path, l4, model_2)

```

```

ax = sns.heatmap([l1, l2, l3, l4], xticklabels=class_names, yticklabels=class_names, annot=True, fmt='g')
ax.set(xlabel='Predicted label', ylabel='True label')
plt.show()

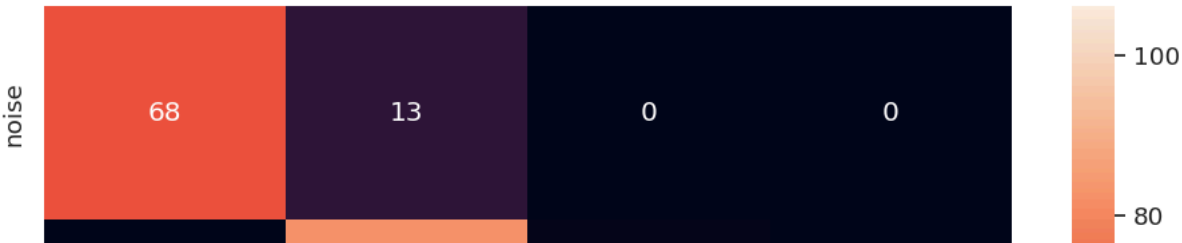
```

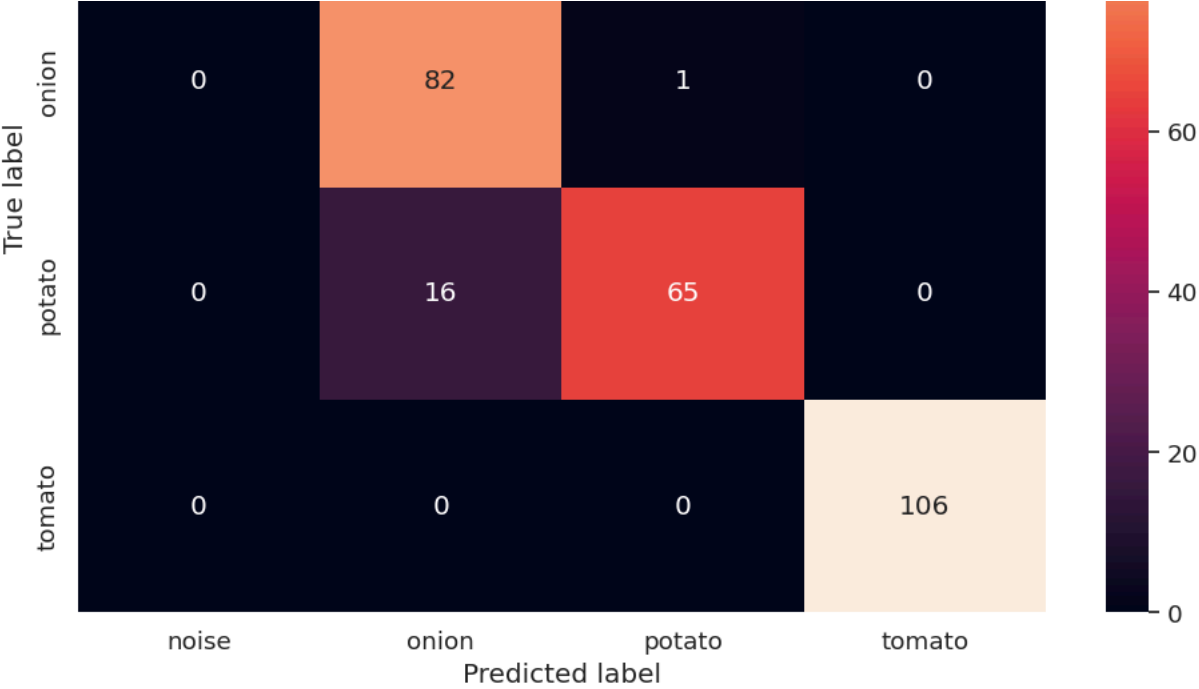
1/1 0s 33ms/step
1/1 0s 45ms/step
1/1 0s 34ms/step
1/1 0s 34ms/step
1/1 0s 34ms/step
1/1 0s 38ms/step
1/1 0s 35ms/step
1/1 0s 38ms/step
1/1 0s 39ms/step
1/1 0s 36ms/step
1/1 0s 35ms/step
1/1 0s 34ms/step
1/1 0s 48ms/step
1/1 0s 37ms/step
1/1 0s 37ms/step
1/1 0s 35ms/step
1/1 0s 35ms/step
1/1 0s 36ms/step
1/1 0s 41ms/step
1/1 0s 34ms/step
1/1 0s 33ms/step
1/1 0s 35ms/step
1/1 0s 34ms/step
1/1 0s 34ms/step
1/1 0s 36ms/step
1/1 0s 36ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 34ms/step
1/1 0s 34ms/step
1/1 0s 32ms/step
1/1 0s 35ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 33ms/step
1/1 0s 35ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 40ms/step
1/1 0s 34ms/step
1/1 0s 34ms/step
1/1 0s 35ms/step
1/1 0s 33ms/step
1/1 0s 41ms/step
1/1 0s 32ms/step
1/1 0s 31ms/step
1/1 0s 33ms/step
1/1 0s 33ms/step
1/1 0s 36ms/step
1/1 0s 38ms/step
1/1 0s 35ms/step
1/1 0s 35ms/step
1/1 0s 36ms/step
1/1 0s 38ms/step
1/1 0s 40ms/step
1/1 0s 36ms/step
1/1 0s 36ms/step
1/1 0s 34ms/step
1/1 0s 32ms/step
1/1 0s 48ms/step
1/1 0s 34ms/step
1/1 0s 34ms/step
1/1 0s 37ms/step
1/1 0s 35ms/step
1/1 0s 32ms/step
1/1 0s 43ms/step
1/1 0s 35ms/step
1/1 0s 31ms/step
1/1 0s 34ms/step
1/1 0s 37ms/step
1/1 0s 49ms/step
1/1 0s 46ms/step
1/1 0s 44ms/step
1/1 0s 52ms/step
1/1 0s 48ms/step
1/1 0s 46ms/step
1/1 0s 51ms/step
1/1 0s 47ms/step
1/1 0s 44ms/step
1/1 0s 42ms/step
1/1 0s 42ms/step
1/1 0s 49ms/step
1/1 0s 52ms/step
1/1 0s 55ms/step
1/1 0s 53ms/step
1/1 0s 68ms/step
1/1 0s 53ms/step
1/1 0s 50ms/step
1/1 0s 47ms/step
1/1 0s 33ms/step

1/1 _____ 0s 35ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 36ms/step

1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 55ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 63ms/step
1/1 _____ 0s 68ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 54ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 55ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 67ms/step
1/1 _____ 0s 53ms/step
1/1 _____ 0s 57ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 50ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 31ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 35ms/step

1/1 _____ 0s 34ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 54ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 54ms/step
1/1 _____ 0s 52ms/step
1/1 _____ 0s 49ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 51ms/step
1/1 _____ 0s 62ms/step
1/1 _____ 0s 56ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 32ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 33ms/step
1/1 _____ 0s 47ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 46ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 35ms/step
1/1 _____ 0s 34ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 38ms/step





```
result = model_2.evaluate(test_ds)
dict(zip(model_2.metrics_names, result))
```

```
11/11 4s 410ms/step - Precision: 0.9197 - Recall: 0.9184 - accuracy: 0.9198 - loss: 0.4365
{'loss': 0.4298263192176819, 'compile_metrics': 0.9173789024353027}
```

Rich text editor toolbar with icons for bold, italic, link, unlink, list, indent, outdent, undo, redo, and others.

#<U>Models Comparison -

Models Comparison -

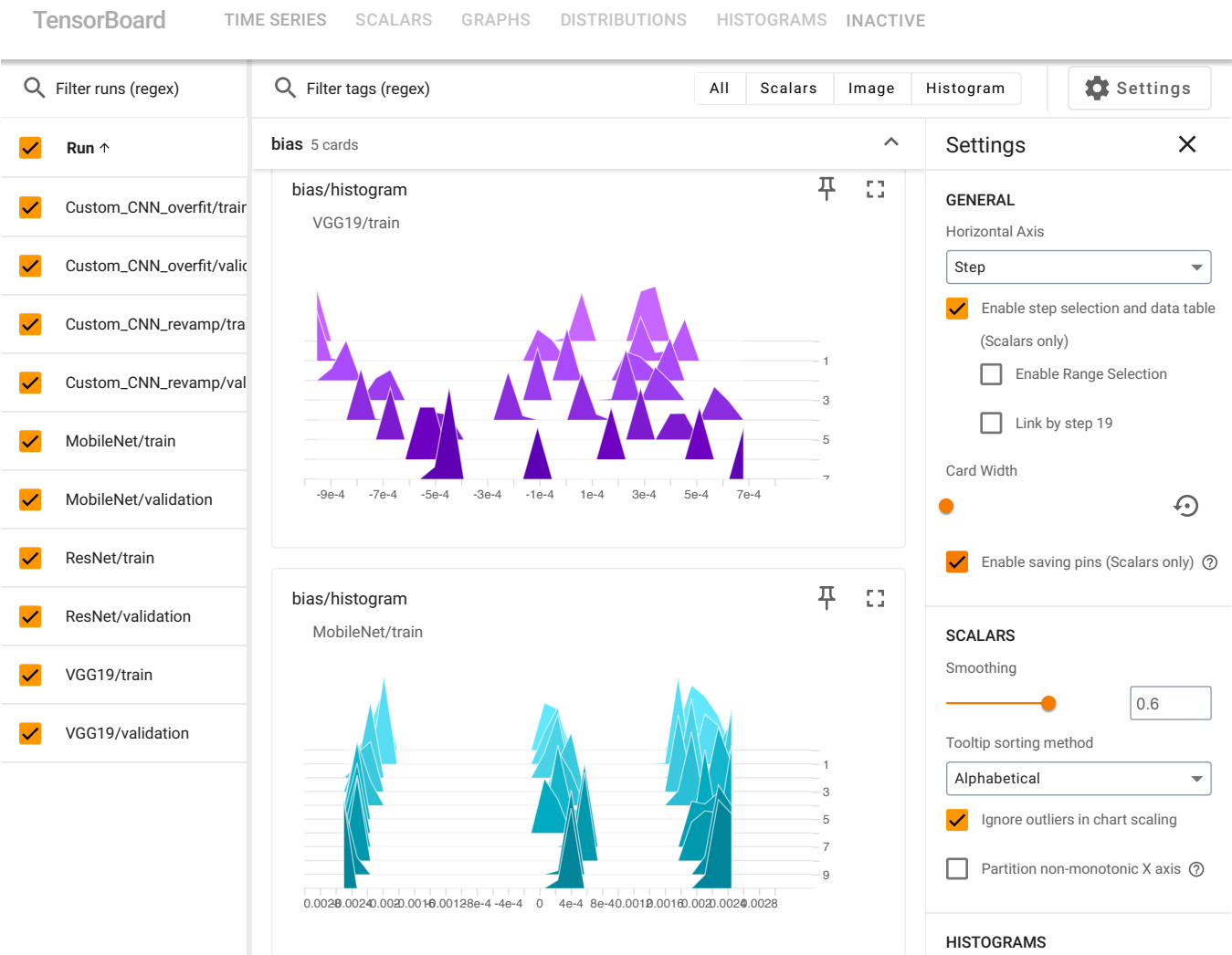
Summary of Trained models:

Model	Test Acc	Train Acc
Custom CNN	76%	81%
CNN Revamp	84%	89%
VGG19	91%	98%
ResNet50	94%	99%
MobileNet	90%	100%

- After training all of our Models, we found Resnet50 to be the most accurate model on our Dataset.
- Let's visualize all of our models via tensorboard from the logs we created during training
- We observed that Resnet50 gave us the highest test accuracy and our baseline CNN gave us the lowest accuracy

```
%load_ext tensorboard
%tensorboard --logdir /content/logs
```

```
The tensorboard extension is already loaded. To reload it, use:
%reload_ext tensorboard
```



✓ **Testing our best model (ResNet)**

ResNet achieved the highest accuracy of **94%** among all the models we trained

Let's test our Finetuned ResNet to predict on some random unseen data to visualize how accurate it is !

```
model_1.summary()
```

🔗 **Model: "sequential_6"**

Layer (type)	Output Shape	Param #
rescaling_5 (Rescaling)	(None, 256, 256, 3)	0
resnet50 (Functional)	(None, 8, 8, 2048)	23,587,712
global_average_pooling2d_5 (GlobalAveragePooling2D)	(None, 2048)	0
dropout_4 (Dropout)	(None, 2048)	0
dense_5 (Dense)	(None, 4)	8,196

Total params: 70,681,486 (269.63 MB)
Trainable params: 23,542,788 (89.81 MB)
Non-trainable params: 53,120 (207.50 KB)
Optimizer params: 47,085,578 (179.62 MB)

```
grid_test_model(model_1)
```

```

1/1 ————— 0s 47ms/step
1/1 ————— 0s 45ms/step
1/1 ————— 0s 48ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 46ms/step
1/1 ————— 0s 46ms/step
1/1 ————— 0s 47ms/step
1/1 ————— 0s 46ms/step

```

prediction : noise

69.0 % noise
28.0 % onion
3.0 % potato
1.0 % tomato

prediction : tomato

0.0 % noise
0.0 % onion
0.0 % potato
100.0 % tomato

prediction : noise

83.0 % noise
16.0 % onion
0.0 % potato
0.0 % tomato

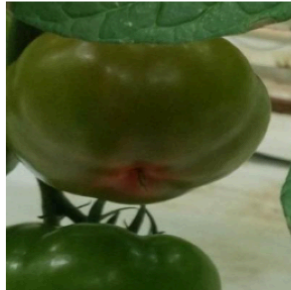
prediction : tomato

0.0 % noise
0.0 % onion
0.0 % potato
100.0 % tomato



prediction : onion

0.0 % noise
93.0 % onion
7.0 % potato
0.0 % tomato



prediction : noise

83.0 % noise
16.0 % onion
2.0 % potato
0.0 % tomato



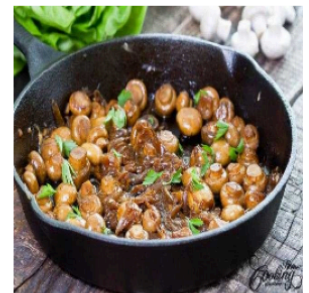
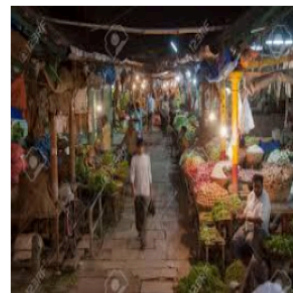
prediction : noise

100.0 % noise
0.0 % onion
0.0 % potato
0.0 % tomato



prediction : onion

0.0 % noise
100.0 % onion
0.0 % potato
0.0 % tomato



We see that our Model is **nearly 100%** sure of the correct class in every image we plotted above, indicating a very good choice for our classification task.

```

result = model_1.evaluate(test_ds)
dict(zip(model_1.metrics_names, result))

```

```

11/11 ————— 6s 539ms/step - Precision: 0.9327 - Recall: 0.9276 - accuracy: 0.9331 - loss: 0.2963
{'loss': 0.2529884874820709, 'compile_metrics': 0.9430199265480042}

```

✓ Key Insights & Observations

1. ResNet50 is the Best Performer Achieved the highest test accuracy (94%) and robust generalization.
Excellent trade-off between performance and overfitting.
Best suited for deployment on your dataset.
2. Custom CNN: A Good Baseline The Custom CNN helped establish a baseline with 76% test accuracy.

Relatively lower accuracy indicates the architecture may lack depth or complexity for this dataset.

Useful for comparison and model improvements.

🔧 3. CNN Revamp Shows Significant Gains Architectural improvements pushed test accuracy to 84%, a +8% improvement over baseline.

Indicates value in iterative tuning of filters, layers, and hyperparameters.

🧠 4. VGG19 and MobileNet are Powerful — but... Both models achieved high train accuracy (98–100%), but:

MobileNet shows possible overfitting (train: 100%, test: 90%)

VGG19 had a slight generalization gap, yet still impressive.

VGG19 is heavy; MobileNet is lighter and better suited for mobile or embedded applications (despite slight overfitting).

📊 5. Visualizations via TensorBoard Training curves in TensorBoard showed:

ResNet50 had a stable and smooth convergence with minimal validation loss.

Custom CNN had noisy learning curves, indicating underfitting or unstable training.

MobileNet and VGG19 trained fast but showed signs of overfitting.

CNN Revamp improved learning dynamics significantly.

✅ Recommendation

Use ResNet50 for deployment or real-world inference.

Optimize MobileNet if speed or resource constraints are a priority.

Keep Custom CNN as a benchmark and use it for ablation studies or experiments.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.